beimingh_Q2

October 3, 2026

1 Initialization

Run the following code to import the modules you'll need. After your finish the assignment, **remember to run all cells** and save the note book to your local machine as a PDF for gradescope submission.

[7]:
```
import os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
```

2 Download data

In this section we will download the data and setup the paths.

[3]:
```
# Download the data
if not os.path.exists('/content/carseq.npy'):
    !wget https://www.cs.cmu.edu/~deva/data/carseq.npy -O /content/carseq.npy
if not os.path.exists('/content/girlseq.npy'):
    !wget https://www.cs.cmu.edu/~deva/data/girlseq.npy -O /content/girlseq.npy
```

```
--2026-10-01 16:59:06--  https://www.cs.cmu.edu/~deva/data/carseq.npy
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 254976128 (243M)
Saving to: '/content/carseq.npy'

/content/carseq.npy 100%[===================>] 243.16M  1.93MB/s    in 2m 10s

2026-10-01 17:01:17 (1.87 MB/s) - '/content/carseq.npy' saved
[254976128/254976128]

--2026-10-01 17:01:17--  https://www.cs.cmu.edu/~deva/data/girlseq.npy
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 27648128 (26M)
```

```
Saving to: '/content/girlseq.npy'

/content/girlseq.np 100%[===================>]  26.37M  1.93MB/s    in 14s

2026-10-01 17:01:31 (1.90 MB/s) - '/content/girlseq.npy' saved
[27648128/27648128]
```

3 Q2.1: Theory Questions (5 points)

Please refer to the handout for the detailed questions.

3.1 Q2.1.1: What is $\frac{\partial \mathbf{W}(\mathbf{x};\mathbf{p})}{\partial \mathbf{p}^T}$? (Hint: It should be a 2x2 matrix)

===== your answer here! =====

$$\frac{\partial \mathbf{W}(\mathbf{x};\mathbf{p})}{\partial \mathbf{p}^T} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

===== end of your answer =====

3.2 Q2.1.2: What is A and b?

===== your answer here! =====

$\mathbf{A}$ is $\nabla I \frac{\partial \mathbf{W}(\mathbf{x};\mathbf{p})}{\partial \mathbf{p}^T}$, $\mathbf{b}$ is $\mathbf{T}(x) - I(\mathbf{W}(\mathbf{x};\mathbf{p}))$

===== end of your answer =====

3.3 Q2.1.3 What conditions must $\mathbf{A}^T\mathbf{A}$ meet so that a unique solution to Δp can be found?

===== your answer here! =====

$\mathbf{A}^T\mathbf{A}$ needs to be invertible.

===== end of your answer =====

4 Q2.2: Lucas-Kanade (20 points)

Make sure to comment your code and use proper names for your variables.

[9]:
```python
from scipy.interpolate import RectBivariateSpline
from numpy.linalg import lstsq

def LucasKanade(It, It1, rect, threshold, num_iters, p0=np.zeros(2)):
    """
    :param[np.array(H, W)] It   : Grayscale image at time t [float]
    :param[np.array(H, W)] It1  : Grayscale image at time t+1 [float]
```

```
    :param[np.array(4, 1)] rect : [x1 y1 x2 y2] coordinates of the rectangular␣
↪template to extract from the image at time t,
                                  where [x1, y1] is the top-left, and [x2, y2]␣
↪is the bottom-right. Note that coordinates
                                  [floats] that maybe fractional.
    :param[float] threshold     : If change in parameters is less than thresh,␣
↪terminate the optimization
    :param[int] num_iters       : Maximum number of optimization iterations
    :param[np.array(2, 1)] p0   : Initial translation parameters [p_x0, p_y0]␣
↪to add to rect, which defaults to [0 0]
    :return[np.array(2, 1)] p   : Final translation parameters [p_x, p_y]
    """

    # Initialize p to p0.
    p = p0

    # ===== your code here! =====
    y_img = np.arange(It.shape[0])
    x_img = np.arange(It.shape[1])

    spline_It = RectBivariateSpline(y_img, x_img, It)
    spline_It1 = RectBivariateSpline(y_img, x_img, It1)

    x = np.arange(rect[0], rect[2] + 1)
    y = np.arange(rect[1], rect[3] + 1)

    x_grid, y_grid = np.meshgrid(x, y)

    tem = spline_It.ev(y_grid, x_grid)

    for itr in range(num_iters):
      x_warped = x_grid + p[0]
      y_warped = y_grid + p[1]
      tem_warped = spline_It1.ev(y_warped, x_warped)

      Ix = spline_It1.ev(y_warped, x_warped, dx=0, dy=1)
      Iy = spline_It1.ev(y_warped, x_warped, dx=1, dy=0)

      Ix = Ix.reshape(-1, 1)
      Iy = Iy.reshape(-1, 1)

      A = np.hstack((Ix, Iy))

      b = (tem - tem_warped).reshape(-1, 1)

      delta_p = lstsq(A, b, rcond=None)[0].flatten()
      p += delta_p
```

```
        if np.linalg.norm(delta_p) < threshold:
            break


    # Hint: Iterate over num_iters and for each iteration, construct a linear␣
↪system (Ax=b) that solves for a x=delta_p update
    # Construct [A] by computing image gradients at (possibly fractional) pixel␣
↪locations.
    # We suggest using RectBivariateSpline from scipy.interpolate to␣
↪interpolate pixel values at fractional pixel locations
    # We suggest using lstsq from numpy.linalg to solve the linear system
    # Once you solve for [delta_p], add it to [p] (and move on to next␣
↪iteration)
    #
    # HINT/WARNING:
    # RectBivariateSpline and Meshgrid use inconsistent defaults with respect␣
↪to 'xy' versus 'ij' indexing:
    # https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.
↪RectBivariateSpline.ev.html#scipy.interpolate.RectBivariateSpline.ev
    # https://numpy.org/doc/stable/reference/generated/numpy.meshgrid.html


    # ===== End of code =====
    return p
```

4.1 Debug Q2.2

A few tips to debug your implementation: - Feel free to use and modify the following snippet to debug your implementation. The snippet simply visualizes the translation resulting from running LK on a single frame. You should be able to see a slight shift in the template.

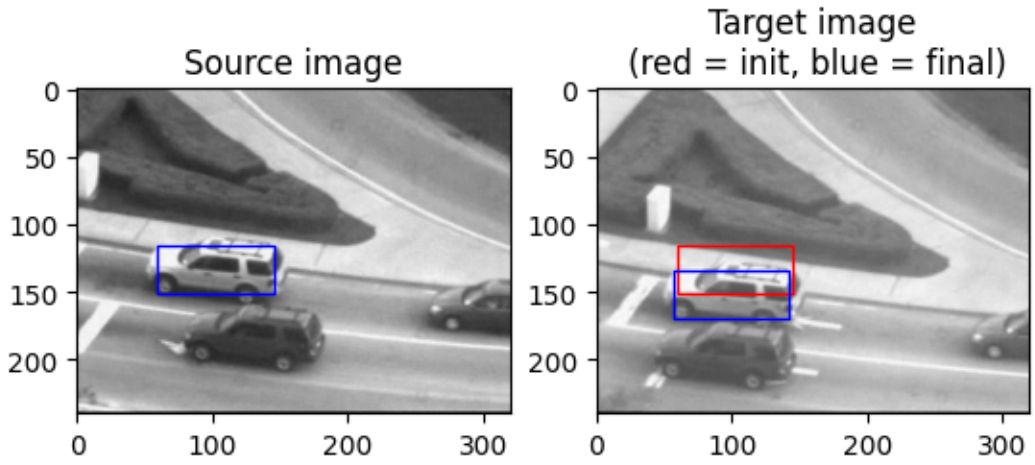
- You may also want to visualize the image gradients you compute within your LK implementation
- Plot iterations vs the norm of delta_p

[10]:
```
def draw_rect(rect,color):
    w = rect[2] - rect[0]
    h = rect[3] - rect[1]
    plt.gca().add_patch(patches.Rectangle((rect[0],rect[1]), w, h, linewidth=1,␣
↪edgecolor=color, facecolor='none'))
```

[11]:
```
num_iters = 100
threshold = 0.01
seq = np.load("/content/carseq.npy")
rect = [59, 116, 145, 151]
It = seq[:,:,0]
```

```
# Source frame
plt.figure()
plt.subplot(1,2,1)
plt.imshow(It, cmap='gray')
plt.title('Source image')
draw_rect(rect,'b')

# Target frame + LK
It1  = seq[:,:, 20]
plt.subplot(1,2,2)
plt.imshow(It1, cmap='gray')
plt.title('Target image\n (red = init, blue = final)')
p = LucasKanade(It, It1, rect, threshold, num_iters, p0=np.zeros(2))
rect_t1 = rect + np.concatenate((p,p))
draw_rect(rect,'r')
draw_rect(rect_t1,'b')
```



4.2 Q2.3: Tracking with template update (15 points)

[24]:
```
def TrackSequence(seq, rect, num_iters, threshold):
    """
    :param seq       : (H, W, T), sequence of frames
    :param rect      : (4, 1), coordinates of template in the initial frame. top-left and bottom-right corners.
    :param num_iters : int, number of iterations for running the optimization
    :param threshold : float, threshold for terminating the LK optimization
    :return: rects   : (T, 4) tracked rectangles for each frame
    """
    H, W, N = seq.shape
```

```
    rects =[]
    It = seq[:,:,0]

    rect = np.array(rect, dtype=float)

    # Iterate over the car sequence and track the car
    rects.append(rect.copy())

    # ===== your code here! =====
    for i in range(N - 1):

        It0 = seq[:, :, i]
        It1 = seq[:, :, i + 1]

        p = LucasKanade(It0,It1,rect,threshold,num_iters,p0=np.zeros(2))

        rect[0] += p[0]
        rect[1] += p[1]
        rect[2] += p[0]
        rect[3] += p[1]

        rects.append(rect.copy())
        # TODO: add your code track the object of interest in the sequence

        # ===== End of code =====

    rects = np.array(rects)
    assert rects.shape == (N, 4), f"Your output sequence {rects.shape} is not␣
↪({N}x{4})"
    return rects
```

4.2.1 Q2.3 (a) - Track Car Sequence

Run the following snippets. If you have implemented LucasKanade and TrackSequence function correctly, you should see the box tracking the car accurately. Please note that the tracking might drift slightly towards the end, and that is entirely normal.

Feel free to play with these snippets of code by playing with the parameters.

[16]:
```
def visualize_track(seq,rects,frames):
    # Visualize tracks on an image sequence for a select number of frames
    plt.figure(figsize=(15,15))
    for i in range(len(frames)):
        idx = frames[i]
        frame = seq[:, :, idx]
        plt.subplot(1,len(frames),i+1)
        plt.imshow(frame, cmap='gray')
```

```
        plt.axis('off')
        draw_rect(rects[idx],'b');
```

[25]:
```
seq = np.load("/content/carseq.npy")
rect = [59, 116, 145, 151]

# NOTE: feel free to play with these parameters
num_iters = 10000
threshold = 0.01

rects = TrackSequence(seq, rect, num_iters, threshold)

visualize_track(seq,rects,[0, 79, 159, 279, 409])
```



4.2.2 Q2.3 (b) - Track Girl Sequence

Same as the car sequence.

[26]:
```
# Loads the squence
seq = np.load("/content/girlseq.npy")
rect = [280, 152, 330, 318]

# NOTE: feel free to play with these parameters
num_iters = 10000
threshold = 0.01

rects = TrackSequence(seq, rect, num_iters, threshold)

visualize_track(seq,rects,[0, 14, 34, 64, 84])
```



4.2.3 Q6 - AI Assistance

1. Did you use any artificial intelligence assistance in solving any part of this assignment?

Answer here: Yes

2. If you answered "Yes", give full details (e.g. "I used Claude to identify and debug errors in my code implementation for Question 2")

Answer here: ChatGPT is used to debug and refine the quality of code.

beimingh_Q3

October 3, 2026

1 Initialization

Run the following code to import the modules you'll need. After your finish the assignment, **remember to run all cells** and save the note book to your local machine as a PDF for gradescope submission.

[1]:
```
import time
import os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
```

2 Download data

In this section we will download the data and setup the paths.

[2]:
```
# Download the data
if not os.path.exists('/content/aerialseq.npy'):
    !wget https://www.cs.cmu.edu/~deva/data/aerialseq.npy -O /content/aerialseq.
 ↪npy
if not os.path.exists('/content/antseq.npy'):
    !wget https://www.cs.cmu.edu/~deva/data/antseq.npy -O /content/antseq.npy
```

```
--2026-10-02 20:03:06--  https://www.cs.cmu.edu/~deva/data/aerialseq.npy
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 92160128 (88M)
Saving to: '/content/aerialseq.npy'

/content/aerialseq. 100%[===================>]  87.89M  1.82MB/s    in 49s

2026-10-02 20:03:57 (1.79 MB/s) - '/content/aerialseq.npy' saved
[92160128/92160128]

--2026-10-02 20:03:57--  https://www.cs.cmu.edu/~deva/data/antseq.npy
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
```

```
HTTP request sent, awaiting response… 200 OK
Length: 65536128 (62M)
Saving to: '/content/antseq.npy'

/content/antseq.npy 100%[===================>]  62.50M  1.87MB/s    in 34s

2026-10-02 20:04:31 (1.83 MB/s) - '/content/antseq.npy' saved
[65536128/65536128]
```

3 Q3: Affine Motion Subtraction

3.1 Q3.1: Dominant Motion Estimation (15 points)

[3]:
```python
from scipy.interpolate import RectBivariateSpline
from numpy.linalg import lstsq

def LucasKanadeAffine(It, It1, threshold, num_iters):
    """
    :param It        : (H, W), current image
    :param It1       : (H, W), next image
    :param threshold : (float), if the length of dp < threshold, terminate the optimization
    :param num_iters : (int), number of iterations for running the optimization

    :return: M       : (2, 3) The affine transform matrix
    """
    # Initial M
    M = np.array([[1.0, 0.0, 0.0], [0.0, 1.0, 0.0]])

    # ===== your code here! =====
    H, W = It.shape
    y_img = np.arange(H)
    x_img = np.arange(W)
    spline_It1 = RectBivariateSpline(y_img, x_img, It1)
    x = np.arange(W)
    y = np.arange(H)
    x_grid, y_grid = np.meshgrid(x, y)
    for itr in range(num_iters):
      x_warped = (M[0, 0] * x_grid + M[0, 1] * y_grid + M[0, 2])
      y_warped = (M[1, 0] * x_grid + M[1, 1] * y_grid + M[1, 2])

      valid = ((x_warped >= 0) & (x_warped <= W - 1) & (y_warped >= 0) & (y_warped <= H - 1))

      xv = x_grid[valid]
      yv = y_grid[valid]
```

```
        xw = x_warped[valid]
        yw = y_warped[valid]

        warped_It1 = spline_It1.ev(yw, xw)

        Ix = spline_It1.ev(yw, xw, dx=0, dy=1)
        Iy = spline_It1.ev(yw, xw, dx=1, dy=0)

        A1 = (Ix * xv).reshape(-1, 1)
        A2 = (Ix * yv).reshape(-1, 1)
        A3 = Ix.reshape(-1, 1)

        A4 = (Iy * xv).reshape(-1, 1)
        A5 = (Iy * yv).reshape(-1, 1)
        A6 = Iy.reshape(-1, 1)

        A = np.hstack((A1, A2, A3, A4, A5, A6))

        b = (It[valid] - warped_It1).reshape(-1, 1)


        delta_p = lstsq(A, b, rcond=None)[0].flatten()

        delta_M = np.array([[delta_p[0], delta_p[1], delta_p[2]], [delta_p[3],␣
    ↪delta_p[4], delta_p[5]]])

        M += delta_M

        if np.linalg.norm(delta_p) < threshold:
            break


    # ===== End of code =====
    return M
```

3.2 Debug Q3.1

Feel free to use and modify the following snippet to debug your implementation. The snippet simply visualizes the translation resulting from running LK on a single frame. When you warp the source frame using the obtained transformation matrix, it should resemble the target frame.

[4]:
```
import cv2

num_iters = 100
threshold = 0.01
seq = np.load("/content/aerialseq.npy")
```

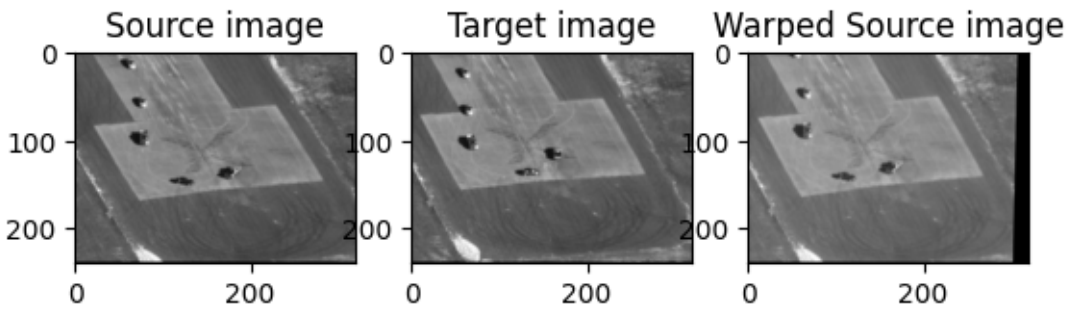
```
It = seq[:,:,0]
It1 = seq[:,:,10]

# Source frame
plt.figure()
plt.subplot(1,3,1)
plt.imshow(It, cmap='gray')
plt.title('Source image')

# Target frame
plt.subplot(1,3,2)
plt.imshow(It1, cmap='gray')
plt.title('Target image')

# Warped source frame
M = LucasKanadeAffine(It, It1, threshold, num_iters)
warped_It = cv2.warpAffine(It, M,(It.shape[1],It.shape[0]))
plt.subplot(1,3,3)
plt.imshow(warped_It, cmap='gray')
plt.title('Warped Source image')
```

[4]: `Text(0.5, 1.0, 'Warped Source image')`



4 Q3.2: Moving Object Detection (10 points)

[5]:
```
import numpy as np
from scipy.ndimage import binary_erosion
from scipy.ndimage import binary_dilation
from scipy.ndimage import affine_transform
import scipy.ndimage
import cv2

def SubtractDominantMotion(It, It1, num_iters, threshold, tolerance):
    """
```

```
    :param It        : (H, W), current image
    :param It1       : (H, W), next image
    :param num_iters : (int), number of iterations for running the optimization
    :param threshold : (float), if the length of dp < threshold, terminate the␣
↪optimization
    :param tolerance : (float), binary threshold of intensity difference when␣
↪computing the mask
    :return: mask    : (H, W), the mask of the moved object
    """
    mask = np.ones(It.shape, dtype=bool)

    # ===== your code here! =====
    M = LucasKanadeAffine(It, It1, threshold, num_iters)
    warped_It = cv2.warpAffine(It,M,(It.shape[1], It.shape[0]))
    mask = np.abs(It1 - warped_It) > tolerance
    mask = binary_erosion(mask)
    mask = binary_dilation(mask)
    # ===== End of code =====

    return mask
```

4.1 Q3.3: Tracking with affine motion (10 points)

[6]:
```
from tqdm import tqdm

def TrackSequenceAffineMotion(seq, num_iters, threshold, tolerance):
    """
    :param seq       : (H, W, T), sequence of frames
    :param num_iters : int, number of iterations for running the optimization
    :param threshold : float, if the length of dp < threshold, terminate the␣
↪optimization
    :param tolerance : (float), binary threshold of intensity difference when␣
↪computing the mask
    :return: masks   : (T, 4) moved objects for each frame
    """
    H, W, N = seq.shape

    masks =[]
    It = seq[:,:,0]

    # ===== your code here! =====
    for i in tqdm(range(1, seq.shape[2])):
        It1 = seq[:, :, i]
        mask = SubtractDominantMotion(It, It1, num_iters, threshold, tolerance)
        masks.append(mask)
        It = It1
```

```
    # ===== End of code =====
    masks = np.stack(masks, axis=2)
    return masks
```

4.2 Q3.3 (a) - Track Ant Sequence

[7]:
```
seq = np.load("/content/antseq.npy")

# NOTE: feel free to play with these parameters
num_iters = 1000
threshold = 0.01
tolerance = 0.2

tic = time.time()
masks = TrackSequenceAffineMotion(seq, num_iters, threshold, tolerance)
toc = time.time()
print('\nAnt Sequence takes %f seconds' % (toc - tic))
```

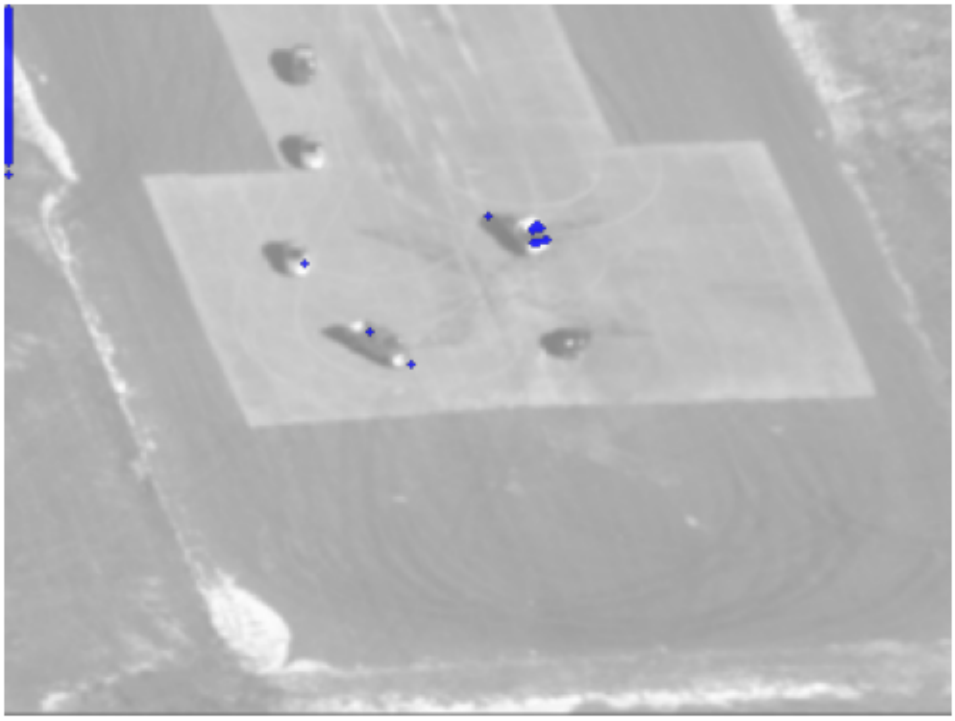
```
100%|      | 124/124 [01:20<00:00,  1.54it/s]

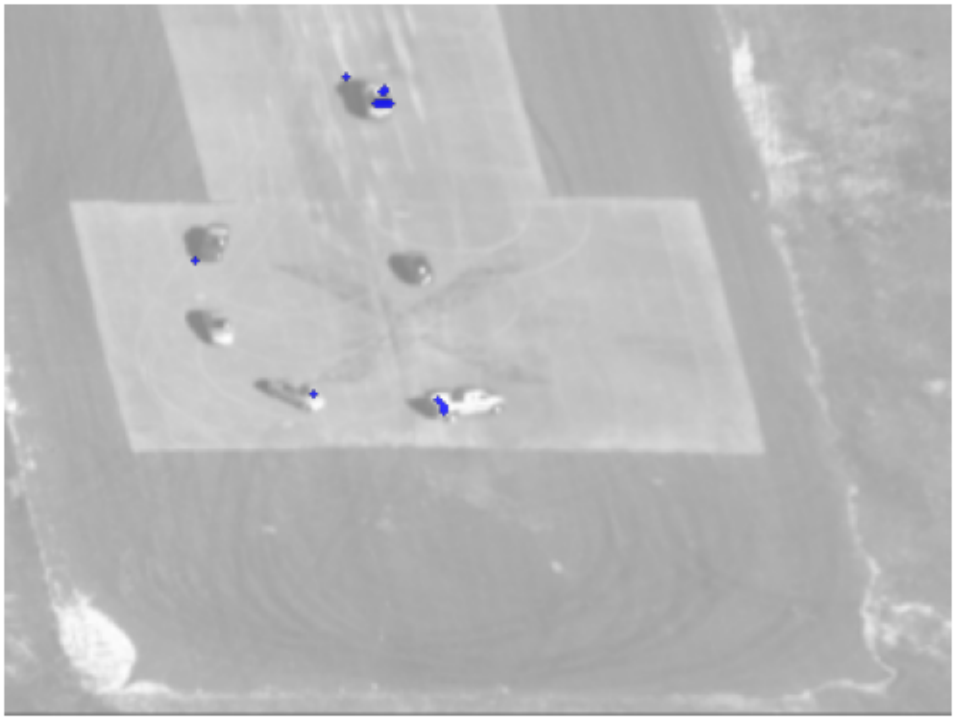

Ant Sequence takes 80.830932 seconds
```

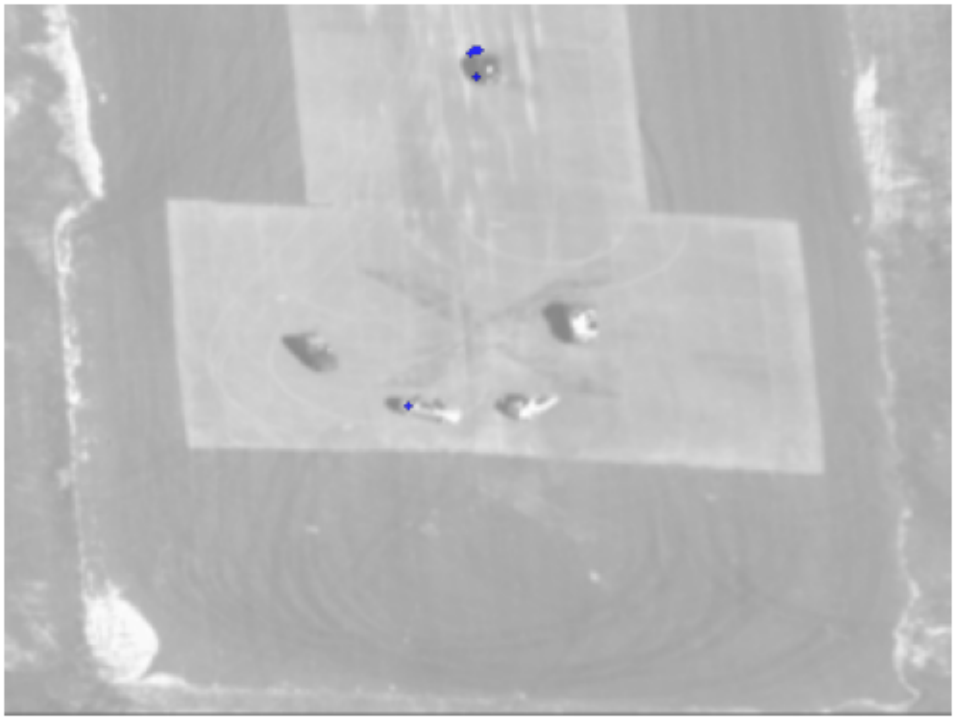
[9]:
```
frames_to_save = [29, 59, 89, 119]

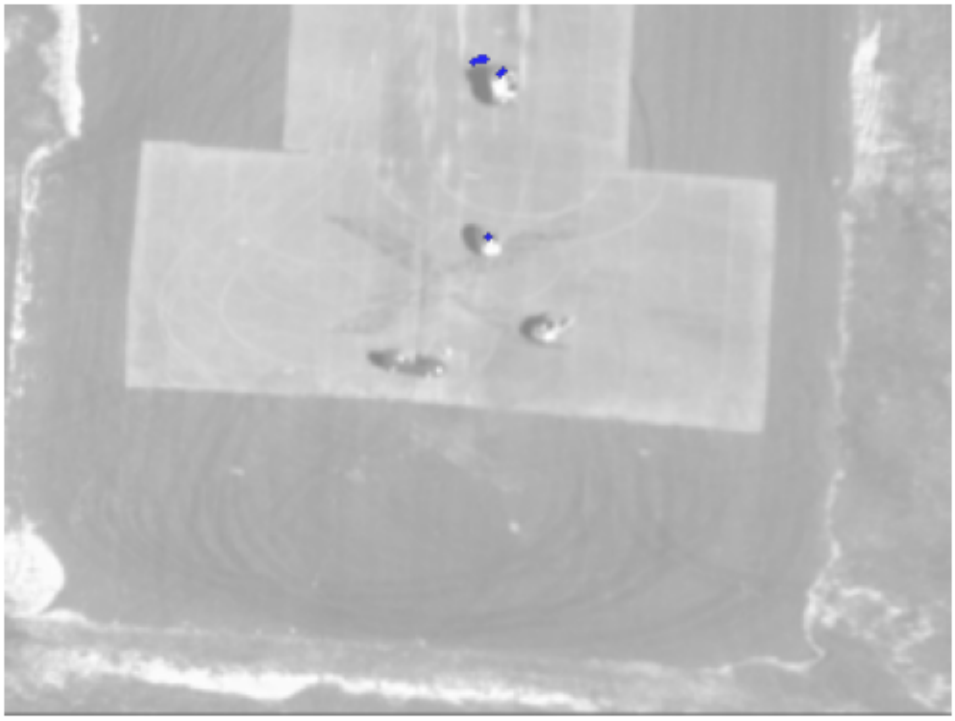
# TODO: visualize
for idx in frames_to_save:
    frame = seq[:, :, idx]
    mask = masks[:, :, idx]

    plt.figure()
    plt.imshow(frame, cmap="gray", alpha=0.5)
    plt.imshow(np.ma.masked_where(np.invert(mask), mask), cmap='winter',␣
 ↪alpha=0.8)
    plt.axis('off')
```









4.2.1 Q3.3 (b) - Track Aerial Sequence

[8]:
```
seq = np.load("/content/aerialseq.npy")

# NOTE: feel free to play with these parameters
num_iters = 1000
threshold = 0.01
tolerance = 0.2

tic = time.time()
masks = TrackSequenceAffineMotion(seq, num_iters, threshold, tolerance)
toc = time.time()
print('\nAnt Sequence takes %f seconds' % (toc - tic))
```

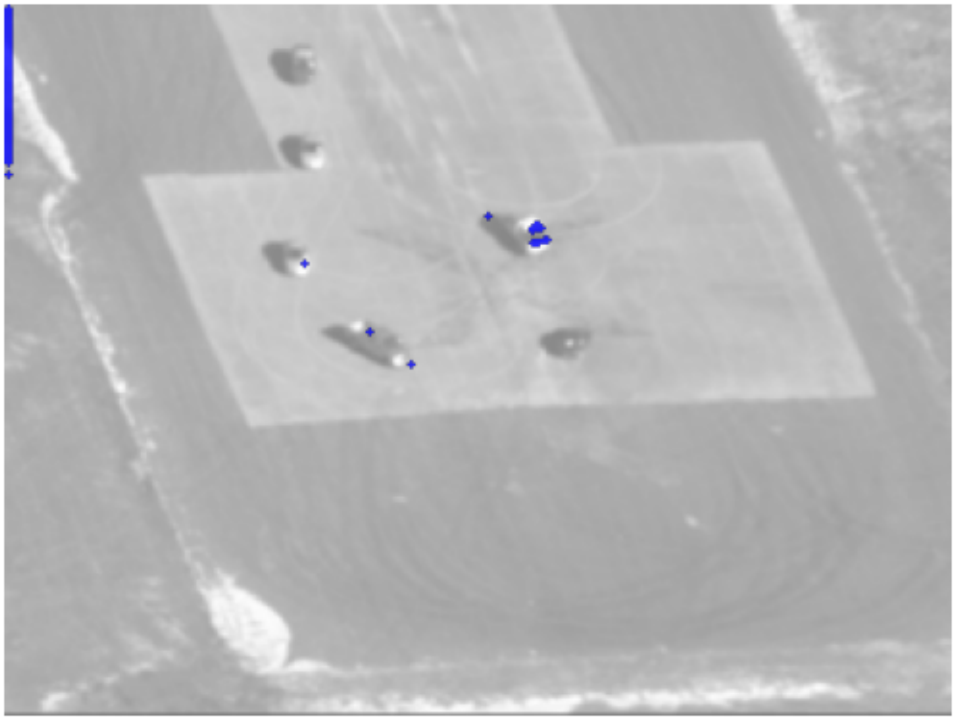
```
100%|      | 149/149 [02:57<00:00,  1.19s/it]

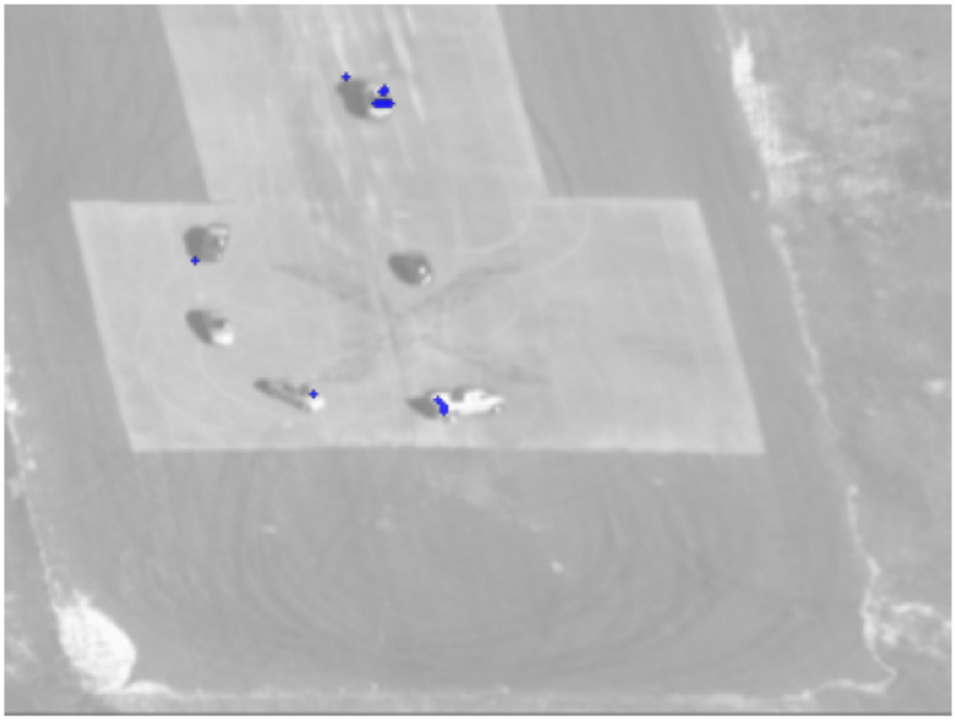
Ant Sequence takes 177.160524 seconds
```

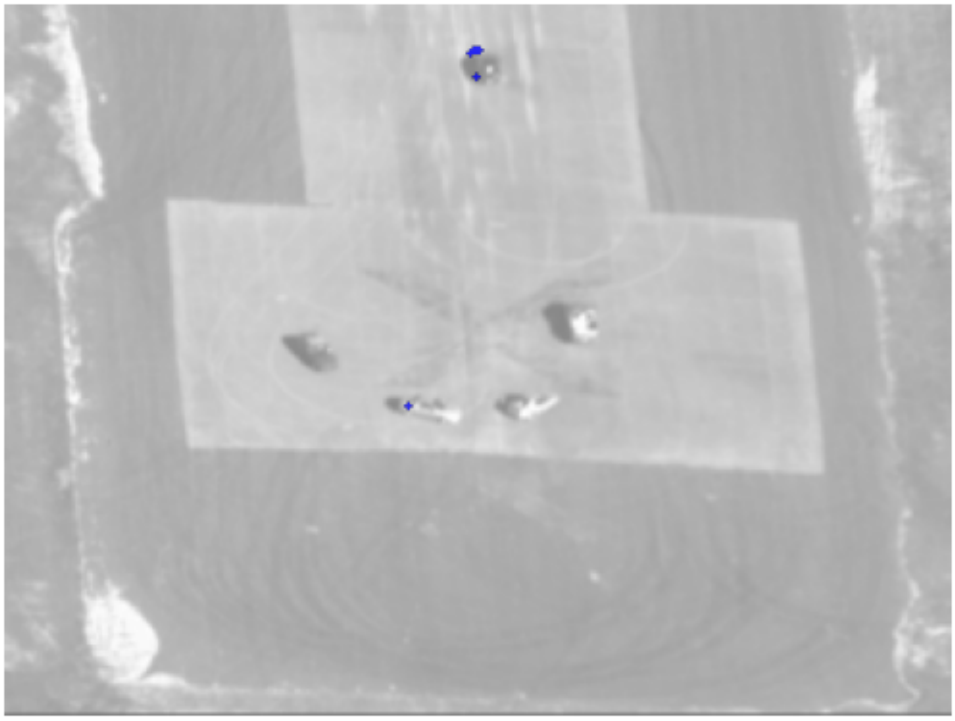
[10]:
```
frames_to_save = [29, 59, 89, 119]

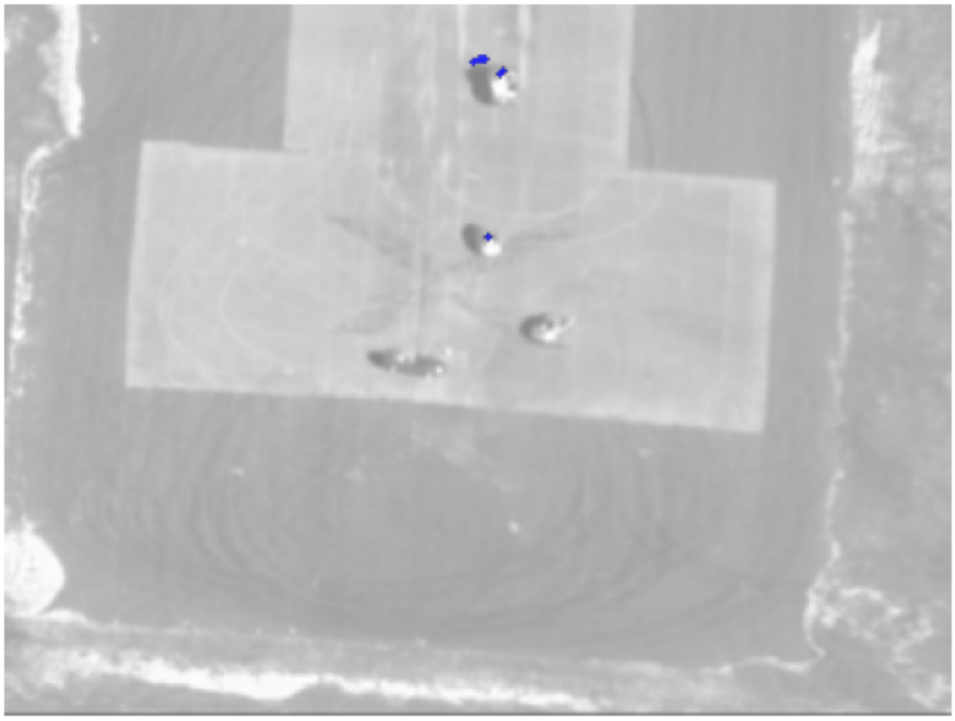
# TODO: visualize
for idx in frames_to_save:
    frame = seq[:, :, idx]
    mask = masks[:, :, idx]

    plt.figure()
    plt.imshow(frame, cmap="gray", alpha=0.5)
    plt.imshow(np.ma.masked_where(np.invert(mask), mask), cmap='winter',␣
 ↪alpha=0.8)
    plt.axis('off')
```









beimingh_Q4

October 3, 2026

1 Initialization

Run the following code to import the modules you'll need. After your finish the assignment, **remember to run all cells** and save the note book to your local machine as a PDF for gradescope submission.

[2]:
```
import time
import os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
```

2 Download data

In this section we will download the data and setup the paths.

[3]:
```
# Download the data
if not os.path.exists('/content/aerialseq.npy'):
    !wget https://www.cs.cmu.edu/~deva/data/aerialseq.npy -O /content/aerialseq.
 ↪npy
if not os.path.exists('/content/antseq.npy'):
    !wget https://www.cs.cmu.edu/~deva/data/antseq.npy -O /content/antseq.npy
```

```
--2026-10-02 00:07:07--  https://www.cs.cmu.edu/~deva/data/aerialseq.npy
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 92160128 (88M)
Saving to: '/content/aerialseq.npy'

/content/aerialseq. 100%[===================>]  87.89M  4.47MB/s    in 20s

2026-10-02 00:07:27 (4.39 MB/s) - '/content/aerialseq.npy' saved
[92160128/92160128]

--2026-10-02 00:07:27--  https://www.cs.cmu.edu/~deva/data/antseq.npy
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
```

```
HTTP request sent, awaiting response… 200 OK
Length: 65536128 (62M)
Saving to: '/content/antseq.npy'

/content/antseq.npy 100%[===================>]  62.50M  4.46MB/s    in 14s

2026-10-02 00:07:42 (4.44 MB/s) - '/content/antseq.npy' saved
[65536128/65536128]
```

3 Q4: Efficient Tracking

3.1 Q4.1: Inverse Composition (15 points)

[4]:
```python
from scipy.interpolate import RectBivariateSpline

def InverseCompositionAffine(It, It1, threshold, num_iters):
    """
    :param It        : (H, W), current image
    :param It1       : (H, W), next image
    :param threshold : (float), if the length of dp < threshold, terminate the optimization
    :param num_iters : (int), number of iterations for running the optimization

    :return: M       : (2, 3) The affine transform matrix
    """
    # Initial M
    M = np.array([[1.0, 0.0, 0.0], [0.0, 1.0, 0.0]])

    # ===== your code here! =====
    H, W = It.shape

    x = np.arange(W)
    y = np.arange(H)
    x_grid, y_grid = np.meshgrid(x, y)
    spline_It = RectBivariateSpline(y, x, It)
    spline_It1 = RectBivariateSpline(y, x, It1)
    Ix = spline_It.ev(y_grid,x_grid,dx=0,dy=1)
    Iy = spline_It.ev(y_grid,x_grid,dx=1,dy=0)
    A1 = (Ix * x_grid).reshape(-1, 1)
    A2 = (Ix * y_grid).reshape(-1, 1)
    A3 = Ix.reshape(-1, 1)
    A4 = (Iy * x_grid).reshape(-1, 1)
    A5 = (Iy * y_grid).reshape(-1, 1)
    A6 = Iy.reshape(-1, 1)
    A = np.hstack((A1, A2, A3, A4, A5, A6))
    template = It.reshape(-1, 1)
```

```
    for itr in range(num_iters):
        x_warped = (M[0, 0] * x_grid + M[0, 1] * y_grid + M[0, 2])
        y_warped = (M[1, 0] * x_grid +M[1, 1] * y_grid +M[1, 2])
        valid = ((x_warped >= 0) & (x_warped <= W - 1) & (y_warped >= 0) &␣
↪(y_warped <= H - 1))
        warped_It1 = spline_It1.ev(y_warped[valid],x_warped[valid]).reshape(-1,␣
↪1)
        valid_flat = valid.reshape(-1)
        A_valid = A[valid_flat]
        template_valid = template[valid_flat]
        b = warped_It1 - template_valid
        delta_p = np.linalg.lstsq(A_valid,b,rcond=None)[0].flatten()
        delta_M = np.array([[1 + delta_p[0], delta_p[1],␣
↪delta_p[2]],[delta_p[3], 1 + delta_p[4], delta_p[5]],[0.0, 0.0, 1.0]])
        M_h = np.vstack(( M, np.array([0.0, 0.0, 1.0])))
        M_h = M_h @ np.linalg.inv(delta_M)

        M = M_h[:2, :]

        if np.linalg.norm(delta_p) < threshold:
            break

    # ===== End of code =====
    return M
```

3.2 Debug Q4.1

Feel free to use and modify the following snippet to debug your implementation. The snippet simply visualizes the translation resulting from running LK on a single frame. When you warp the source frame using the obtained transformation matrix, it should resemble the target frame.

[5]:
```
import cv2

num_iters = 100
threshold = 0.01
seq = np.load("/content/aerialseq.npy")
It = seq[:,:,0]
It1 = seq[:,:,10]

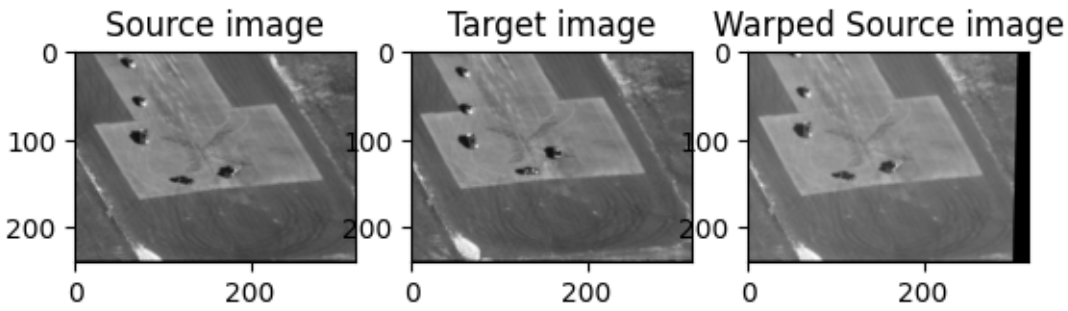
# Source frame
plt.figure()
plt.subplot(1,3,1)
plt.imshow(It, cmap='gray')
plt.title('Source image')

# Target frame
```

```
plt.subplot(1,3,2)
plt.imshow(It1, cmap='gray')
plt.title('Target image')

# Warped source frame
M = InverseCompositionAffine(It, It1, threshold, num_iters)
warped_It = cv2.warpAffine(It, M,(It.shape[1],It.shape[0]))
plt.subplot(1,3,3)
plt.imshow(warped_It, cmap='gray')
plt.title('Warped Source image')
```

[5]: `Text(0.5, 1.0, 'Warped Source image')`



3.3 Q4.2 Tracking with Inverse Composition (10 points)

Re-use your impplementation in Q3.2 for subtract dominant motion. Just make sure to use InverseCompositionAffine within.

[6]:
```
import numpy as np
from scipy.ndimage import binary_erosion
from scipy.ndimage import binary_dilation
from scipy.ndimage import affine_transform
import scipy.ndimage
import cv2

def SubtractDominantMotion(It, It1, num_iters, threshold, tolerance):
    """
    :param It        : (H, W), current image
    :param It1       : (H, W), next image
    :param num_iters : (int), number of iterations for running the optimization
    :param threshold : (float), if the length of dp < threshold, terminate the␣
 ↪optimization
    :param tolerance : (float), binary threshold of intensity difference when␣
 ↪computing the mask
    :return: mask    : (H, W), the mask of the moved object
```

```
    """
    mask = np.ones(It.shape, dtype=bool)

    # ===== your code here! =====
    M = InverseCompositionAffine(It, It1, threshold, num_iters)
    warped_It = cv2.warpAffine(It,M,(It.shape[1], It.shape[0]))
    mask = np.abs(It1 - warped_It) > tolerance
    mask = binary_erosion(mask)
    mask = binary_dilation(mask)
    # ===== End of code =====

    return mask
```

Re-use your implementation in Q3.3 for sequence tracking.

[9]:
```
from tqdm import tqdm

def TrackSequenceAffineMotion(seq, num_iters, threshold, tolerance):
    """
    :param seq       : (H, W, T), sequence of frames
    :param num_iters : int, number of iterations for running the optimization
    :param threshold : float, if the length of dp < threshold, terminate the␣
 ↪optimization
    :param tolerance : (float), binary threshold of intensity difference when␣
 ↪computing the mask
    :return: masks   : (T, 4) moved objects for each frame
    """
    H, W, N = seq.shape

    masks =[]
    It = seq[:,:,0]

    # ===== your code here! =====
    for i in tqdm(range(1, seq.shape[2])):
        It1 = seq[:, :, i]
        mask = SubtractDominantMotion(It, It1, num_iters, threshold, tolerance)
        masks.append(mask)
        It = It1

    # ===== End of code =====
    masks = np.stack(masks, axis=2)
    return masks
```

Track the ant sequence with inverse composition method.

[18]:
```
seq = np.load("/content/antseq.npy")

# NOTE: feel free to play with these parameters
```

```
num_iters = 1000
threshold = 0.01
tolerance = 0.2

tic = time.time()
masks = TrackSequenceAffineMotion(seq, num_iters, threshold, tolerance)
toc = time.time()
print('\nAnt Sequence takes %f seconds' % (toc - tic))
```

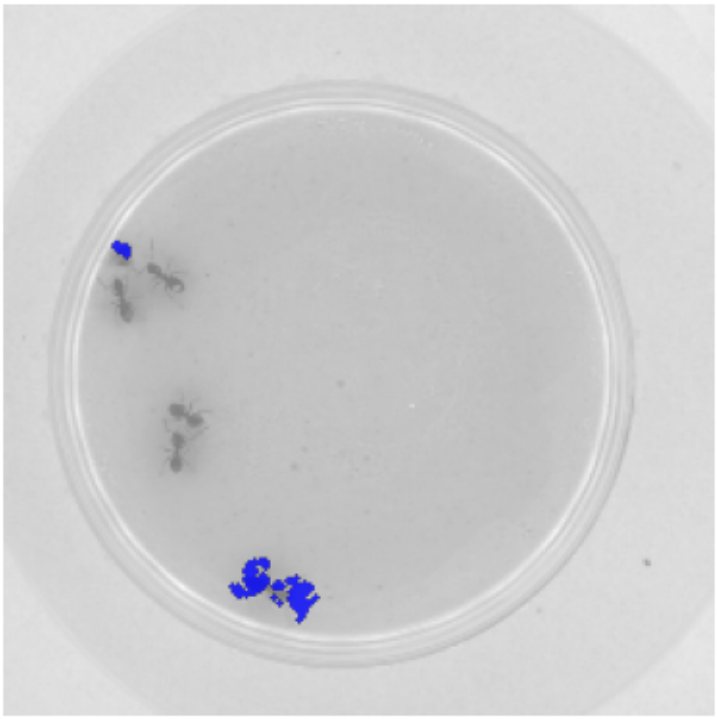
```
100%|      | 124/124 [00:38<00:00,  3.22it/s]

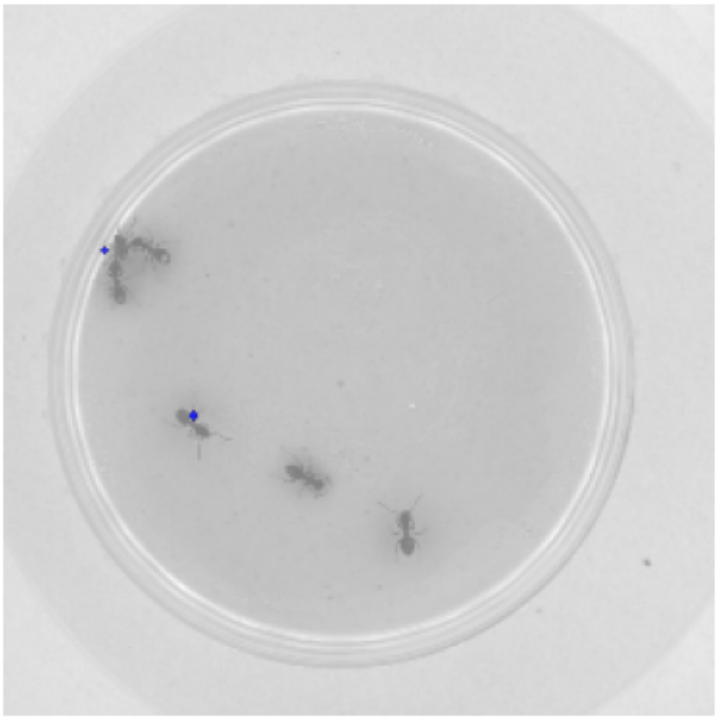

Ant Sequence takes 38.592302 seconds
```

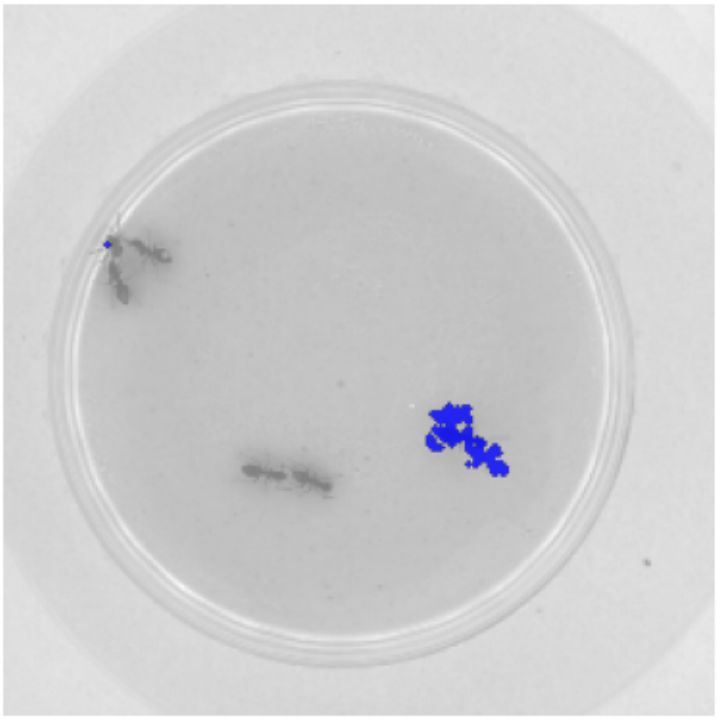
[15]:
```
frames_to_save = [29, 59, 89, 119]

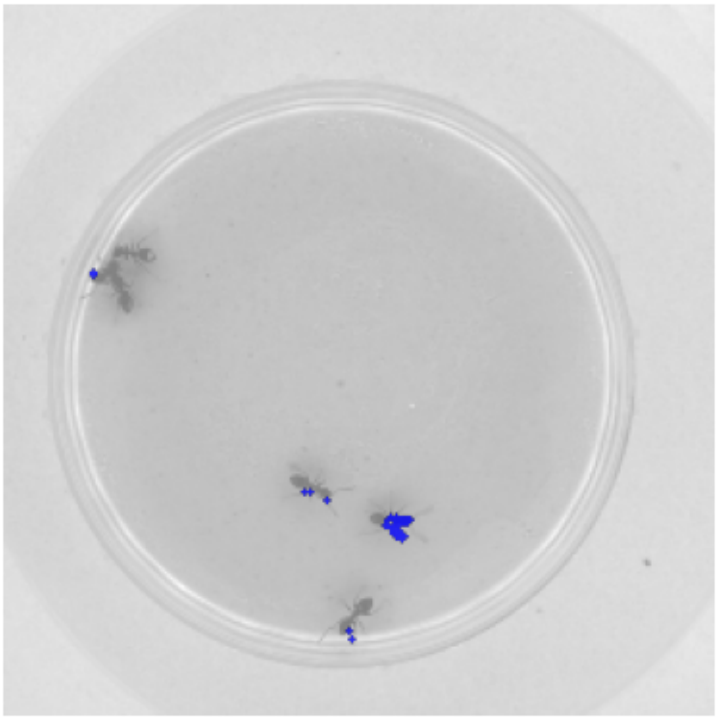
# TODO: visualize
for idx in frames_to_save:
    frame = seq[:, :, idx]
    mask = masks[:, :, idx]

    plt.figure()
    plt.imshow(frame, cmap="gray", alpha=0.5)
    plt.imshow(np.ma.masked_where(np.invert(mask), mask), cmap='winter',␣
 ↪alpha=0.8)
    plt.axis('off')
```









Track the aerial sequence with inverse composition method.

[19]:
```
seq = np.load("/content/aerialseq.npy")

# NOTE: feel free to play with these parameters
num_iters = 1000
threshold = 0.01
tolerance = 0.2

tic = time.time()
masks = TrackSequenceAffineMotion(seq, num_iters, threshold, tolerance)
toc = time.time()
print('\nAnt Sequence takes %f seconds' % (toc - tic))
```

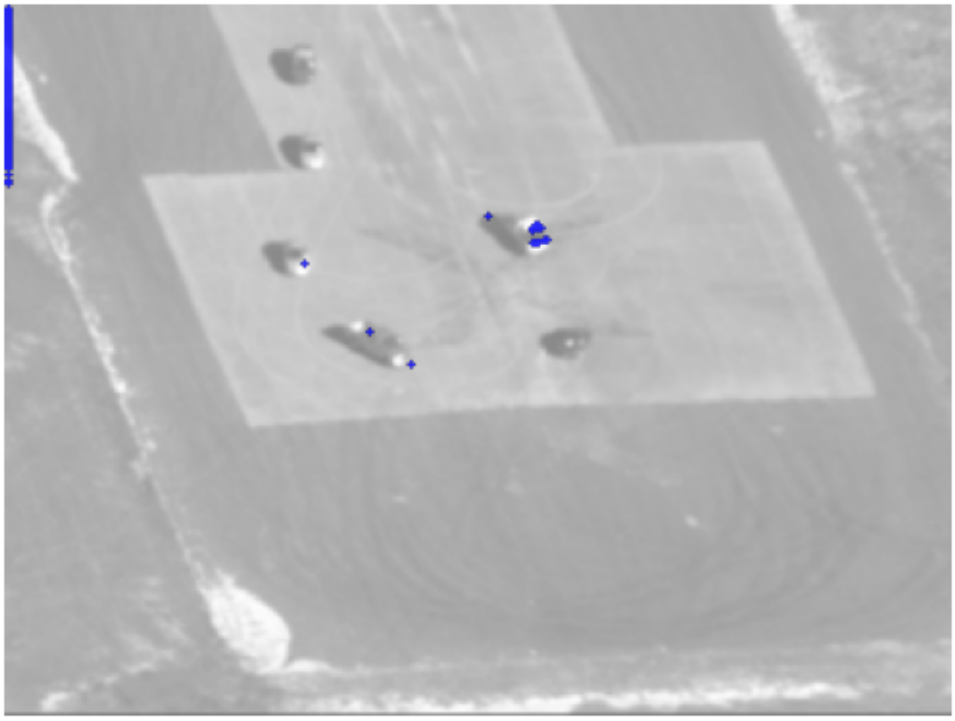
```
100%|      | 149/149 [03:05<00:00,  1.24s/it]

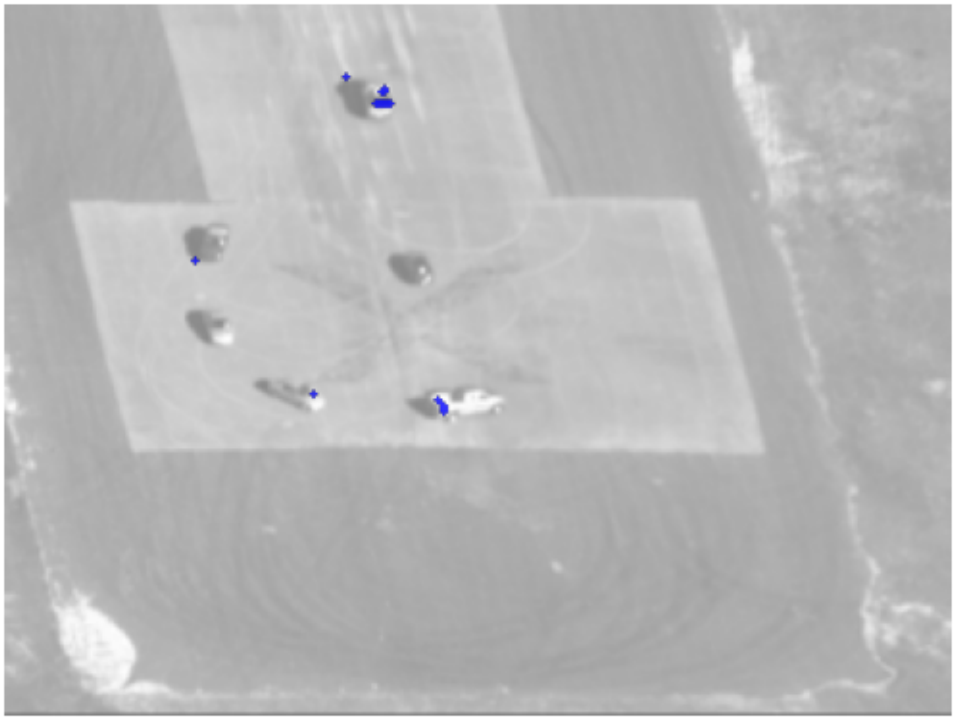

Ant Sequence takes 185.391699 seconds
```

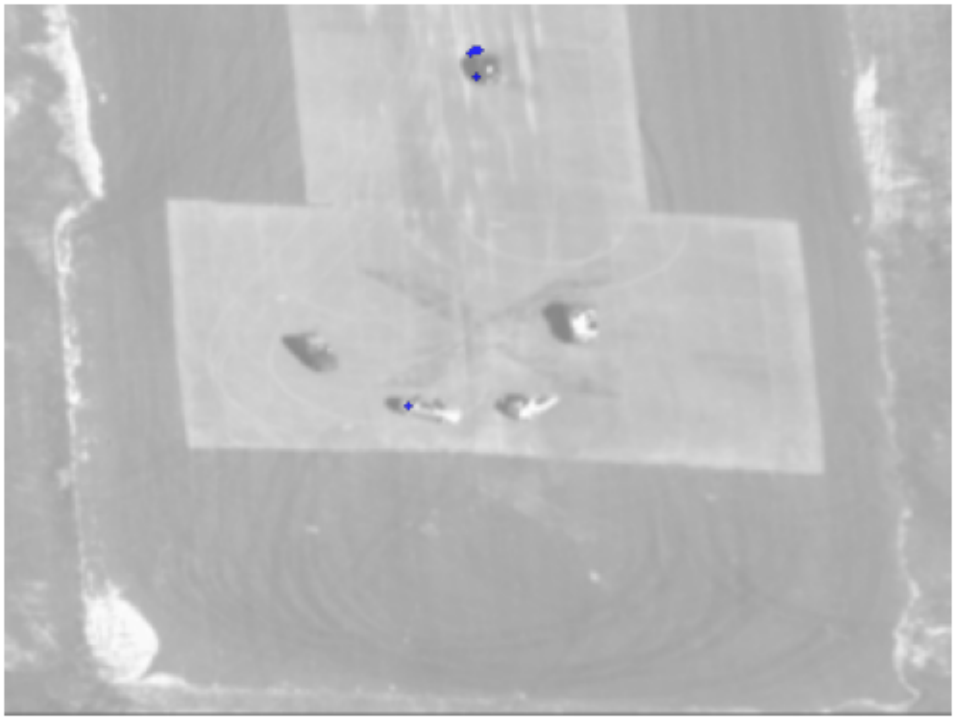
[20]:
```
frames_to_save = [29, 59, 89, 119]

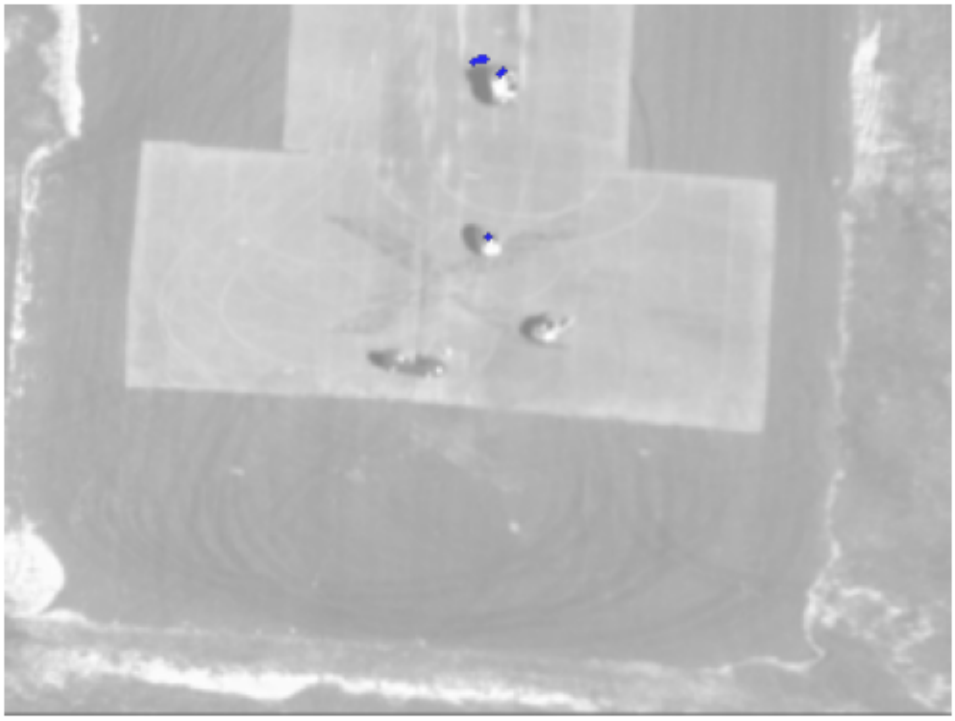
# TODO: visualize
for idx in frames_to_save:
    frame = seq[:, :, idx]
    mask = masks[:, :, idx]

    plt.figure()
    plt.imshow(frame, cmap="gray", alpha=0.5)
    plt.imshow(np.ma.masked_where(np.invert(mask), mask), cmap='winter',␣
 ↪alpha=0.8)
    plt.axis('off')
```









3.4 Q4.2.1 Compare the runtime of the algorithm using inverse composition (as described in this section) with its runtime without inverse composition (as detailed in the previous section) in the context of the ant and aerial sequences:

===== your answer here! =====

The inverse compositional implementation should run faster than the classical forward-additive Lucas–Kanade affine algorithm. The exact runtimes depend on the number of iterations required for convergence, but the inverse compositional method reduces the amount of computation performed inside each iteration.

===== end of your answer ====

3.5 Q4.2.2 In your own words, please describe briefly why the inverse compositional approach is more computationally efficient than the classical approach:

===== your answer here! =====

The template does not change during the optimization, these quantities can be computed once before the iteration loop and reused.

===== end of your answer ====

beimingh_Q5

October 3, 2026

1 Initialization

Run the following code to import the modules you'll need. After your finish the assignment, **remember to run all cells** and save the note book to your local machine as a PDF for gradescope submission.

[1]:
```
import os
import numpy as np
import matplotlib.pyplot as plt
```

2 Download data

In this section we will download the data and setup the paths.

[2]:
```
# Download the data
if not os.path.exists('/content/bead_data.npz'):
    !wget https://www.cs.cmu.edu/~deva/data/bead_data.npz -O /content/bead_data.
 ↪npz

if not os.path.exists('/content/hammer_handle_data.npz'):
    !wget https://www.cs.cmu.edu/~deva/data/hammer_handle_data.npz -O /content/
 ↪hammer_handle_data.npz
```

```
--2026-10-02 18:53:40--  https://www.cs.cmu.edu/~deva/data/bead_data.npz
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 138394388 (132M)
Saving to: '/content/bead_data.npz'

/content/bead_data. 100%[===================>] 131.98M  4.91MB/s    in 27s

2026-10-02 18:54:07 (4.91 MB/s) - '/content/bead_data.npz' saved
[138394388/138394388]

--2026-10-02 18:54:07--
https://www.cs.cmu.edu/~deva/data/hammer_handle_data.npz
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
```

```
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 92698388 (88M)
Saving to: '/content/hammer_handle_data.npz'

/content/hammer_han 100%[===================>]  88.40M  5.11MB/s    in 17s

2026-10-02 18:54:25 (5.08 MB/s) - '/content/hammer_handle_data.npz' saved
[92698388/92698388]
```

We provide the following helper function, which computes the rotation matrix from the parameterization θ and its derivative with respect to rotation component of θ.

[10]:
```
"""
Helper Functions
"""

def get_dR_dtheta_xyz(theta_x, theta_y, theta_z):
    """
    Compute parital derivatives of the rotation matrix R = Rz(theta_z) @␣
 ↪Ry(theta_y) @ Rx(theta_x) wrt theta_x, theta_y, theta_z.

    :param[float] theta_x               : x element of x-y-z Euler Rotation.
    :param[float] theta_y               : y element of x-y-z Euler Rotation.
    :param[float] theta_z               : z element of x-y-z Euler Rotation.
    :return
        [np.array (3, 3)] dR_dtheta_x   : The partial derivative of the␣
 ↪rotation matrix with regards to theta_x.
        [np.array (3, 3)] dR_dtheta_y   : The partial derivative of the␣
 ↪rotation matrix with regards to theta_y.
        [np.array (3, 3)] dR_dtheta_z   : The partial derivative of the␣
 ↪rotation matrix with regards to theta_z.
    """
    cx, cy, cz = np.cos(theta_x), np.cos(theta_y), np.cos(theta_z)
    sx, sy, sz = np.sin(theta_x), np.sin(theta_y), np.sin(theta_z)

    # Rotation matrices
    Rx = np.array([
        [1, 0, 0],
        [0, cx, -sx],
        [0, sx, cx]
    ])
    Ry = np.array([
        [cy, 0, sy],
        [0, 1, 0],
        [-sy, 0, cy]
    ])
```

```
    Rz = np.array([
        [cz, -sz, 0],
        [sz,  cz, 0],
        [0,   0,  1]
    ])

    # Derivatives of Rx wrt x
    dRx_dx = np.array([
        [0, 0, 0],
        [0, -sx, -cx],
        [0, cx, -sx]
    ])
    # Derivatives of Ry wrt y
    dRy_dy = np.array([
        [-sy, 0, cy],
        [0, 0, 0],
        [-cy, 0, -sy]
    ])
    # Derivatives of Rz wrt z
    dRz_dz = np.array([
        [-sz, -cz, 0],
        [cz, -sz, 0],
        [0, 0, 0]
    ])

    # Chain rule for full R = Rz * Ry * Rx
    dR_dx = Rz @ Ry @ dRx_dx
    dR_dy = Rz @ dRy_dy @ Rx
    dR_dz = dRz_dz @ Ry @ Rx

    return dR_dx, dR_dy, dR_dz

def get_R(theta_x, theta_y, theta_z):
    """
    Compute the rotation matrix R = Rz(theta_z) @ Ry(theta_y) @ Rx(theta_x).

    :param[float] theta_x        : x element of x-y-z Euler Rotation.
    :param[float] theta_y        : y element of x-y-z Euler Rotation.
    :param[float] theta_z        : z element of x-y-z Euler Rotation.
    :return[np.array (3, 3)] R   : The rotation matrix.
    """
    cx, cy, cz = np.cos(theta_x), np.cos(theta_y), np.cos(theta_z)
    sx, sy, sz = np.sin(theta_x), np.sin(theta_y), np.sin(theta_z)

    # Rotation matrices
    Rx = np.array([
        [1, 0, 0],
```

```
        [0, cx, -sx],
        [0, sx, cx]
    ])
    Ry = np.array([
        [cy, 0, sy],
        [0, 1, 0],
        [-sy, 0, cy]
    ])
    Rz = np.array([
        [cz, -sz, 0],
        [sz,  cz, 0],
        [0,   0,  1]
    ])
    R = Rz @ Ry @ Rx
    return R
```

3 Q5.2: Implement NormalFlow (15 points)

Make sure to comment your code and use proper names for your variables.

[3]:
```
from scipy.interpolate import RectBivariateSpline
from numpy.linalg import lstsq

def NormalFlow(It, It1, Ht, Ht1, Ct, Ct1, max_iters=20, rot_threshold=0.0001,␣
 ↪trans_threshold=0.1, init_theta=np.zeros(5)):
    """
    :param[np.array(H, W, 3)] It   : Normal Map at time t [float]
    :param[np.array(H, W, 3)] It1  : Normal Map at time t+1 [float]
    :param[np.array(H, W)] Ht      : Height Map at time t, unit: pixel [float]
    :param[np.array(H, W)] Ht1     : Height Map at time t+1, unit: pixel [float]
    :param[np.array(H, W)] Ct      : Contact Mask at time t [bool]
    :param[np.array(H, W)] Ct1     : Contact Mask at time t+1 [bool]
    :param[int] max_iters          : Max number of optimization iterations
    :param[float] rot_threshold    : If change in rotation parameters is less␣
 ↪than thresh, terminate the optimization
    :param[float] trans_threshold  : If change in translation parameters is␣
 ↪less than thresh, terminate the optimization
    :param[np.array(5)] init_theta : The initial parameter
    :return[np.array(4, 4)] t1_T_t : The Homogeneous transformation matrix of␣
 ↪the object from frame t to frame t+1
    """
    # Initialize theta to zeros
    theta = init_theta.copy()

    # ===== your code here! =====
    H, W, N = It.shape
```

```
    x = np.arange(W)
    y = np.arange(H)
    x_grid, y_grid = np.meshgrid(x, y)
    points = np.stack((x_grid, y_grid, Ht), axis=2)
    spline_It1 = []
    for c in range(3):
        spline_It1.append(RectBivariateSpline(y, x, It1[:, :, c]))

    spline_Ct1 = RectBivariateSpline(y, x, Ct1.astype(float), kx=1, ky=1)
    for itr in range(max_iters):
        theta_x, theta_y, theta_z, tx, ty = theta
        R = get_R(theta_x, theta_y, theta_z)
        dR_dx, dR_dy, dR_dz = get_dR_dtheta_xyz(theta_x, theta_y, theta_z)
        dR = [dR_dx, dR_dy, dR_dz]
        warped_points = points @ R.T

        x_warped = warped_points[:, :, 0] + tx
        y_warped = warped_points[:, :, 1] + ty
        valid = ((x_warped >= 0) & (x_warped <= W - 1) & (y_warped >= 0) &␣
↪(y_warped <= H - 1))
        warped_contact = np.zeros((H, W), dtype=bool)
        warped_contact[valid] = (spline_Ct1.ev(y_warped[valid],␣
↪x_warped[valid]) > 0.5)
        shared_contact = Ct.astype(bool) & warped_contact & valid

        if np.sum(shared_contact) == 0:
            break
        xw = x_warped[shared_contact]
        yw = y_warped[shared_contact]
        pts = points[shared_contact]

        normal_t = It[shared_contact]

        N = pts.shape[0]

        normal_t1 = np.zeros((N, 3))
        grad_x = np.zeros((N, 3))
        grad_y = np.zeros((N, 3))

        for c in range(3):
            normal_t1[:, c] = spline_It1[c].ev(yw, xw)
            grad_x[:, c] = spline_It1[c].ev(yw, xw, dx=0, dy=1)
            grad_y[:, c] = spline_It1[c].ev(yw, xw, dx=1, dy=0)

        rotated_normal = normal_t @ R.T
        A_columns = []
        for j in range(3):
```

```
            dpoint = pts @ dR[j].T

            dx_warped = dpoint[:, 0]
            dy_warped = dpoint[:, 1]

            dnormal = normal_t @ dR[j].T
            A_j = (grad_x * dx_warped[:, None] + grad_y * dy_warped[:, None] -␣
↪dnormal)
            A_columns.append(A_j.reshape(-1, 1))

        A_tx = grad_x.reshape(-1, 1)
        A_columns.append(A_tx)
        A_ty = grad_y.reshape(-1, 1)
        A_columns.append(A_ty)

        A = np.hstack(A_columns)

        b = (rotated_normal - normal_t1).reshape(-1, 1)

        delta_theta = lstsq(A, b, rcond=None)[0].flatten()

        theta += delta_theta

        if (np.linalg.norm(delta_theta[:3]) < rot_threshold and np.linalg.
↪norm(delta_theta[3:]) < trans_threshold):
            break
    # Hint: Iterate over max_iters and for each iteration, construct a linear␣
↪system (Ax=b) that solves for a x=delta_theta update
    # Construct [A] by computing image gradients at (possibly fractional) pixel␣
↪locations and for each channel
    # It might be easier to consider compute the gradient over each element of␣
↪theta independently
    # For each iteration, we suggest to first compute the warped pixel, then␣
↪compute the shared contact mask, finally construct the linear system
    # We suggest using RectBivariateSpline from scipy.interpolate to␣
↪interpolate pixel values at fractional pixel locations
    # We suggest using lstsq from numpy.linalg to solve the linear system
    # Once you solve for [delta_theta], add it to [theta] (and move on to next␣
↪iteration)
    # Use the following termination condition:
    #
    # if (np.linalg.norm(delta_theta[:3]) < rot_threshold and np.linalg.
↪norm(delta_theta[3:]) < trans_threshold):
    #     break
    #
    # HINT/WARNING:
```

```
    # RectBivariateSpline and Meshgrid use inconsistent defaults with respect␣
↪to 'xy' versus 'ij' indexing:
    # https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.
↪RectBivariateSpline.ev.html#scipy.interpolate.RectBivariateSpline.ev
    # https://numpy.org/doc/stable/reference/generated/numpy.meshgrid.html


    # ===== End of code =====
    return theta
```

3.1 Debug Q5.2

Debugging NormalFlow can be challenging. We provide the ground-truth object pose (in θ) between frame 90 and frame 0 in the code below. You can use this to verify your implementation. Below are some tips for debugging:

- **Warping function**: Plot $I_{t+1}(\mathbf{W}(x;\theta))$ and $\mathbf{R}_\theta I_t(x)$, masked with the shared mask $\bar{C}$. If your implementation is correct, the two should match. A helper function below is provided to convert a normal map into an RGB image.
- **Gradient of the warped image**: Once you have a correct implementation of $I_{t+1}(\mathbf{W}(x;\theta))$, compute its numerical derivative $(I_{t+1}(\mathbf{W}(x;\theta+d\theta)) - I_{t+1}(\mathbf{W}(x;\theta)))/d\theta$ for each dimension of θ. Plot the results and check if they match your implementation of $\nabla I_{t+1}(\mathbf{W}(x;\theta))$.
- **Gradient of the rotated image**: Use the same procedure to debug your implementation of $\nabla \mathbf{R}_\theta I_t(x)$.
- **Convergence with provided initialization**: If you use the provided θ as `init_theta`, NormalFlow should converge very quickly to a value very close to the ground-truth θ.
- **Convergence plot**: Plot the iterations versus the norm of `delta_theta`. Your algorithm should show fast convergence.

[4]:
```
"""
Helper Functions
"""

def normal2image(I):
    """
    Given a normal map, turn it into an image and return that

    :param[np.array (H, W, 3)] I        : The Normal Map
    :return[np.array (H, W, 3)] image   : The image
    """
    image = (np.clip((I + 1.0) / 2.0, 0.0, 1.0) * 255).astype(np.uint8)
    return image
```

[5]:
```
# load the data
data_path = "/content/bead_data.npz"
```

```
data = np.load(data_path, allow_pickle=True)
Cs = data["contact_masks"]
Is = data["normal_maps"]
Hs = data["height_maps"]

# We offer you the true object pose (in theta) of frame 90 relevant to frame 0
It = Is[0]; Ct = Cs[0]; Ht = Hs[0]
It1 = Is[90]; Ct1 = Cs[90]; Ht1 = Hs[90]
true_theta = [-0.00875, -0.00544, 0.15949, 43.419, -49.342]

# Plot the images
plt.imshow(normal2image(It))
plt.axis('off')
plt.show()
plt.imshow(normal2image(It1))
plt.axis('off')
plt.show()
```





3.2 Q5.3: Tracking with NormalFlow (10 points)

Implement the tracking method that, given a sequence of tactile-derived local geometries, returns the object pose of each frame relative to the first frame in the form of a 4×4 homogeneous transformation matrix. A helper function is provided to convert θ into a homogeneous matrix. For convenience, this helper assumes zero translation along the z-axis, since the object remains in contact with the sensor.

[6]:
```
"""
Helper Functions
"""

def theta2T(theta):
    """
    Given the theta, assume zero z-translation, compute homogeneous transform␣
 ↪matrix.

    :param[np.array (5)] theta        : The parameterization of the object␣
 ↪transformation.
    :return[np.array (4, 4)] T        : The homogeneous transformation matrix.
    """
    T = np.eye(4, dtype=np.float32)
    T[:3, :3] = get_R(theta[0], theta[1], theta[2])
```

```
    T[0, 3] = theta[3]
    T[1, 3] = theta[4]
    return T
```

[7]:
```
def TrackSequence(Is, Hs, Cs):
    """
    :param[np.array(N, H, W, 3)] Is    : Normal Map sequence [float]
    :param[np.array(N, H, W)] Hs       : Height Map sequence, unit: pixel [float]
    :param[np.array(N, H, W)] Cs       : Contact Mask sequence [bool]
    :param[int] max_iters              : Max number of optimization iterations
    :param[float] rot_threshold        : If change in rotation parameters is␣
↪less than thresh, terminate the optimization
    :param[float] trans_threshold      : If change in translation parameters is␣
↪less than thresh, terminate the optimization
    :return:[np.array(N, 4, 4)] Ts     : object pose with regards to the first␣
↪frame in the form of homogeneous transformation
    """
    init_T = np.eye(4, dtype=np.float32)
    Ts = [init_T.copy()]
    # Iterate over the car sequence and track the car
    for i in range(Is.shape[0] - 1):

        # ===== your code here! =====
        theta = NormalFlow(Is[i], Is[i + 1], Hs[i], Hs[i + 1],Cs[i], Cs[i + 1])
        relative_T = theta2T(theta)
        R = relative_T[:3, :3]
        tx = theta[3]
        ty = theta[4]

        H, W = Hs[i].shape

        x = np.arange(W)
        y = np.arange(H)
        x_grid, y_grid = np.meshgrid(x, y)

        points = np.stack((x_grid, y_grid, Hs[i]),axis=2)
        warped_points = points @ R.T
        x_warped = warped_points[:, :, 0] + tx
        y_warped = warped_points[:, :, 1] + ty
        valid = ((x_warped >= 0) & (x_warped <= W - 1) & (y_warped >= 0) &␣
↪(y_warped <= H - 1))
        height_spline = RectBivariateSpline(y, x, Hs[i + 1])
        contact_spline = RectBivariateSpline(y, x, Cs[i + 1].astype(float),␣
↪kx=1, ky=1)

        target_contact = np.zeros((H, W), dtype=bool)
```

```
        target_contact[valid] = (contact_spline.ev(y_warped[valid], x_warped[valid]) > 0.5)
        shared_contact = Cs[i].astype(bool) & target_contact & valid

        if np.any(shared_contact):
            target_height = height_spline.ev(y_warped[shared_contact], x_warped[shared_contact])
            predicted_height = warped_points[:, :, 2][shared_contact]
            tz = np.mean(target_height - predicted_height)
        else:
            tz = 0.0

        relative_T[2, 3] = tz
        current_T = relative_T @ Ts[-1]
        Ts.append(current_T)

        # ===== End of code =====

    Ts = np.stack(Ts, 0)
    assert Ts.shape == (Is.shape[0], 4, 4), f"Your output sequence {Ts.shape} is not ({Is.shape[0]}x{4}x{4})"
    return Ts
```

3.2.1 Q5.3 (a) - Track Bead Sequence

Run the snippets below. If your implementations of NormalFlow and TrackSequence are correct, each normal image will show the correct estimated 6DoF object pose with an annotated coordinate frame. Feel free to experiment with the parameters in the code.

[8]:
```
"""
Helper Functions
"""

def display_normal_with_coordinate_system(I, T, center_ref, title, vector_scale=50):
    """
    Show the normal map annotating with the computed coordinate system
    :param[np.array(H, W, 3)] I     : Normal Map [float]
    :param[np.array(4, 4)] T        : The homogeneous transformation matrix
    :param[np.array(2)] center_ref  : The x-y origin of the reference frame (first frame)
    :param[float] vector_scale      : The size of the coordinate axis arrow
    """
    fig, ax = plt.subplots(figsize=(3,3))
    ax.imshow(normal2image(I))

    # Get the center and the unit_vector
```

```
    unit_vectors_ref = np.eye(3)[:, :2]
    center_3d_ref = np.array([(center_ref[0] - I.shape[1] / 2 + 0.5),␣
 ↪(center_ref[1] - I.shape[0] / 2 + 0.5), 0])
    unit_vectors_3d_ref = np.eye(3)
    remapped_center_3d_ref = (
        np.dot(T[:3, :3], center_3d_ref) + T[:3, 3]
    )
    remapped_cx_ref = remapped_center_3d_ref[0] + I.shape[1] / 2 - 0.5
    remapped_cy_ref = remapped_center_3d_ref[1] + I.shape[0] / 2 - 0.5
    remapped_center_ref = np.array([remapped_cx_ref, remapped_cy_ref]).astype(
        np.int32
    )
    remapped_unit_vectors_ref = (
        np.dot(T[:3, :3], unit_vectors_3d_ref.T).T
    )[:, :2]

    # Annotate the origin
    ax.scatter(remapped_center_ref[0], remapped_center_ref[1], c="k", alpha=1,␣
 ↪s=10, zorder=3)

    # Annotate the three axes
    colors = ["r", "g", "b"]
    for i in range(3):
        dx, dy = remapped_unit_vectors_ref[i] * vector_scale
        ax.arrow(
            remapped_center_ref[0], remapped_center_ref[1], dx, dy,
            head_width=6, head_length=6,
            fc=colors[i], ec=colors[i],
            linewidth=1, length_includes_head=True
        )

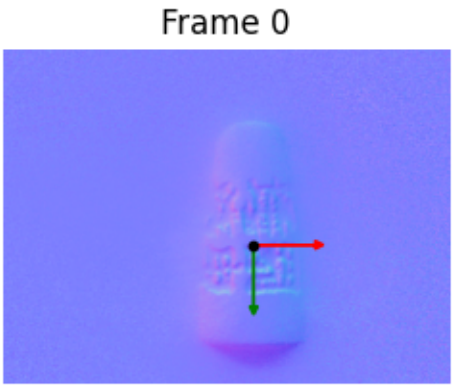
    ax.set_axis_off()
    ax.set_title(title)
    plt.show()

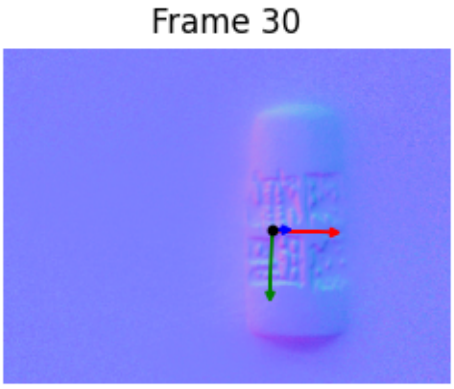
def visualize_track(Is, Ts, vis_seq=[0]):
    # Visualize tracks on a seuqence of a selected sequence
    ys, xs = np.nonzero(Cs[0])
    cx_ref = xs.mean()
    cy_ref = ys.mean()
    center_ref = np.array([cx_ref, cy_ref]).astype(np.int32)
    for idx in vis_seq:
        display_normal_with_coordinate_system(Is[idx], Ts[idx], center_ref,␣
 ↪"Frame %d"%idx)
```

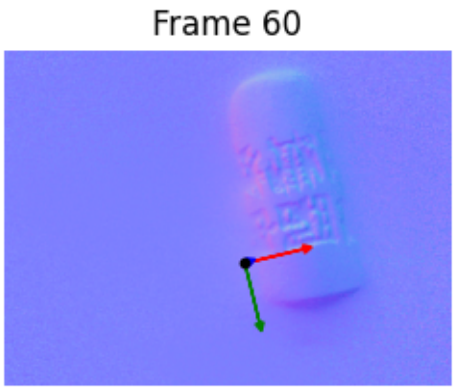
[11]:
```
# load the data
data_path = "/content/bead_data.npz"
```

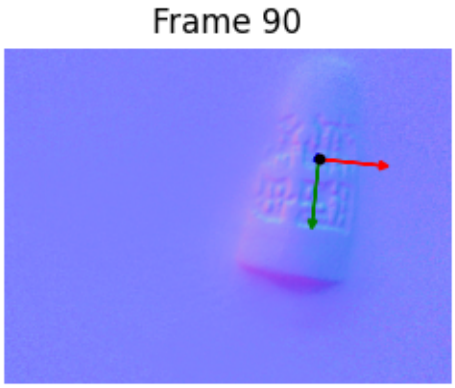
```
data = np.load(data_path, allow_pickle=True)
Cs = data["contact_masks"]
Is = data["normal_maps"]
Hs = data["height_maps"]

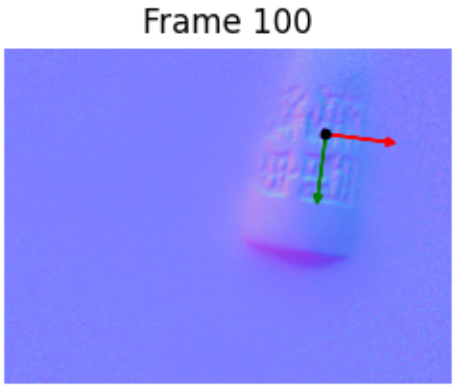
# Track the Bead
Ts = TrackSequence(Is, Hs, Cs)
# Visualize the tracking result
visualize_track(Is, Ts, [0, 30, 60, 90, 100])
```









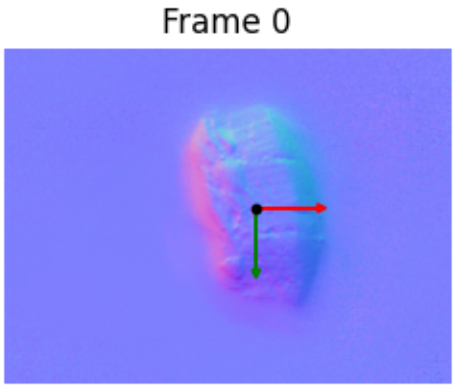


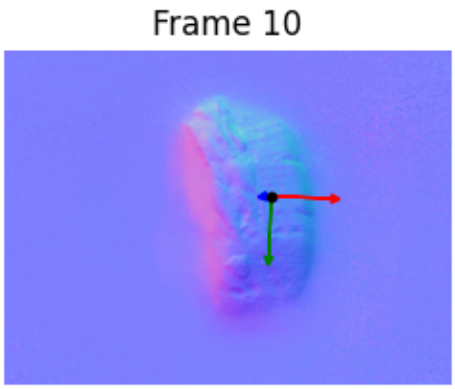
3.2.2 Q5.3 (b) - Track Hammer Handle Sequence

Track the handle of the hammer, which is made by wood.

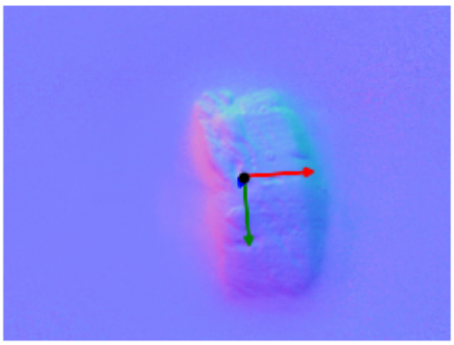
[12]:
```
# load the data
data_path = "/content/hammer_handle_data.npz"
data = np.load(data_path, allow_pickle=True)
Cs = data["contact_masks"]
Is = data["normal_maps"]
Hs = data["height_maps"]

# Track the Hammer Handle
Ts = TrackSequence(Is, Hs, Cs)
# Visualize the tracking result
visualize_track(Is, Ts, [0, 10, 30, 60, 70])
```

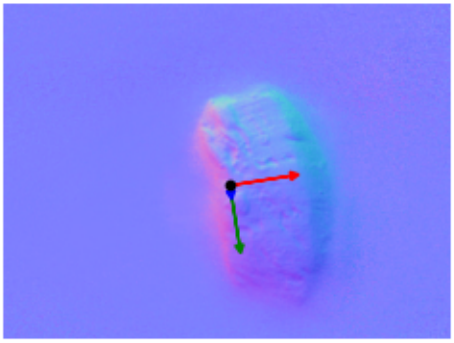




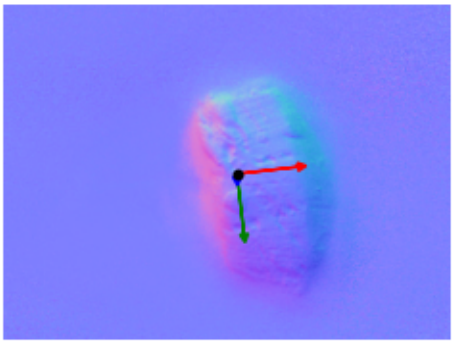
Frame 30



Frame 60



Frame 70



beimingh_Q6

October 3, 2026

1 Q6: Tracking with Segment Anything 2 (Extra Credit, 15 points)

In this notebook you will track the **same objects, starting from the same initial rectangles** as in `LucasKanade.ipynb` (Q2.3), but with a modern pre-trained video segmentation model, SAM 2, instead of Lucas-Kanade. You only run the model (inference); there is no training. Section 6.1 of the handout explains how the model works; please refer to the handout for the detailed questions.

Before you start: switch to a GPU runtime (`Runtime -> Change runtime type -> T4 GPU`). The notebook also runs on CPU, but the car sequence then takes roughly 15-20 minutes instead of about one.

After you finish the assignment, **remember to run all cells** and save the notebook to your local machine as a PDF for gradescope submission.

2 Initialization

SAM 2 video support was added to Hugging Face `transformers` in version 4.56, so we first make sure a recent enough version is installed (this is a no-op on a recent Colab image).

[1]:
```
!pip install -q "transformers>=4.56" accelerate
```

[2]:
```
import os
import time
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import torch
from tqdm import tqdm

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print('Running on:', device)
if device == 'cpu':
    print('WARNING: no GPU found. Tracking will work but be slow. '
          'Use Runtime -> Change runtime type -> T4 GPU.')
```

```
Running on: cuda
```

3 Download data

Same two sequences as in Q2.3.

[3]:
```
# Download the data
if not os.path.exists('/content/carseq.npy'):
    !wget https://www.cs.cmu.edu/~deva/data/carseq.npy -O /content/carseq.npy
if not os.path.exists('/content/girlseq.npy'):
    !wget https://www.cs.cmu.edu/~deva/data/girlseq.npy -O /content/girlseq.npy
```

```
--2026-10-02 19:36:27--  https://www.cs.cmu.edu/~deva/data/carseq.npy
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 254976128 (243M)
Saving to: '/content/carseq.npy'

/content/carseq.npy 100%[===================>] 243.16M  1.37MB/s    in 3m 4s

2026-10-02 19:39:32 (1.32 MB/s) - '/content/carseq.npy' saved
[254976128/254976128]

--2026-10-02 19:39:32--  https://www.cs.cmu.edu/~deva/data/girlseq.npy
Resolving www.cs.cmu.edu (www.cs.cmu.edu)… 128.2.42.95
Connecting to www.cs.cmu.edu (www.cs.cmu.edu)|128.2.42.95|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 27648128 (26M)
Saving to: '/content/girlseq.npy'

/content/girlseq.np 100%[===================>]  26.37M  1.38MB/s    in 19s

2026-10-02 19:39:53 (1.36 MB/s) - '/content/girlseq.npy' saved
[27648128/27648128]
```

4 Helper functions

`draw_rect` is copied from `LucasKanade.ipynb`. `visualize_masks` paints the predicted mask on top of each frame, and `to_rgb_frame` turns a grayscale frame into the 3-channel image SAM 2 expects.

[4]:
```
def draw_rect(rect, color):
    w = rect[2] - rect[0]
    h = rect[3] - rect[1]
    plt.gca().add_patch(patches.Rectangle((rect[0], rect[1]), w, h,␣
 ↪linewidth=1, edgecolor=color, facecolor='none'))
```

```
def visualize_masks(seq, masks, frames, prompt_rect=None):
    '''
    Show the frames listed in `frames` with the SAM 2 mask painted in␣
 ↪translucent blue.
    If prompt_rect is given, it is drawn in red on frame 0 (it is the box␣
 ↪prompt).
    '''
    plt.figure(figsize=(15, 15))
    for i, idx in enumerate(frames):
        plt.subplot(1, len(frames), i + 1)
        plt.imshow(seq[:, :, idx], cmap='gray')
        overlay = np.zeros(masks[idx].shape + (4,))
        overlay[masks[idx]] = [0.1, 0.4, 1.0, 0.45]    # RGBA
        plt.imshow(overlay)
        title = f'frame {idx}'
        if prompt_rect is not None and idx == 0:
            draw_rect(prompt_rect, 'r')
            title += '\n(red = box prompt)'
        plt.axis('off')
        plt.title(title)


def to_rgb_frame(gray):
    '''
    Convert one grayscale frame (H, W) from the .npy sequences into the (H, W,␣
 ↪3) uint8 RGB
    image that SAM 2 expects. Works for both [0, 1] float and [0, 255] inputs.
    '''
    g = np.asarray(gray, dtype=np.float64)
    if g.max() <= 1.0:
        g = g * 255.0
    g = np.clip(np.round(g), 0, 255).astype(np.uint8)
    return np.stack([g, g, g], axis=-1)
```

5 Q6.2: Tracking with SAM 2 (15 points)

Make sure to comment your code and use proper names for your variables.

We load the smallest SAM 2.1 checkpoint (`hiera-tiny`, about 39M parameters). It is plenty for 240x320 frames. If you are curious, `facebook/sam2.1-hiera-small` is a bit more accurate and a bit slower; larger checkpoints are not needed here.

[5]:
```
from transformers import Sam2VideoModel, Sam2VideoProcessor

MODEL_ID = 'facebook/sam2.1-hiera-tiny'

processor = Sam2VideoProcessor.from_pretrained(MODEL_ID)
```

```
model = Sam2VideoModel.from_pretrained(MODEL_ID).to(device)
model.eval()
print(f'{MODEL_ID}: {sum(p.numel() for p in model.parameters()) / 1e6:.1f}M␣
  ↪parameters')
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
```

```
processor_config.json:   0%|          | 0.00/95.0 [00:00<?, ?B/s]
```

```
preprocessor_config.json:   0%|          | 0.00/683 [00:00<?, ?B/s]
```

```
video_preprocessor_config.json:   0%|          | 0.00/705 [00:00<?, ?B/s]
```

```
config.json:   0%|          | 0.00/5.70k [00:00<?, ?B/s]
```

```
[transformers] `memory_attention_rope_theta` is deprecated and will be removed
in v5.0. Use `rope_parameters['rope_theta']` instead.
```

```
model.safetensors: reconstructing file:   0%|          |  0.00B /  156MB
```

```
model.safetensors: downloading bytes:           |  0.00B
```

```
Loading weights:   0%|          | 0/469 [00:00<?, ?it/s]
```

```
facebook/sam2.1-hiera-tiny: 39.0M parameters
```

5.1 How the streaming API works

SAM 2 keeps everything it remembers about the video (the prompt, and the features and masks of the recent frames) inside an **inference session** object. We will use it in *streaming* mode, which means we hand it one frame at a time, exactly like the loop over frames in `TrackSequence` (Q2.3). The four calls you need are:

```
# once, before the loop
inference_session = processor.init_video_session(inference_device=device)

# every frame: resize/normalize the frame for the network
inputs = processor(images=frame_rgb, return_tensors='pt').to(device)

# first frame only: tell the model which object to track, using our rect as a box prompt
processor.add_inputs_to_inference_session(
    inference_session=inference_session,
    frame_idx=0,
    obj_ids=1,                                  # any integer id for the object
    input_boxes=[[[x1, y1, x2, y2]]],           # shape: [batch][object][4]
    original_size=inputs.original_sizes[0],     # required in streaming mode
)

# every frame: run the network. This also writes the frame into the memory bank.
```

```
out = model(inference_session=inference_session, frame=inputs.pixel_values[0])

# every frame: out.pred_masks holds low-resolution per-pixel scores (logits).
# post_process_masks upsamples them to the frame size and thresholds them at 0.
masks = processor.post_process_masks([out.pred_masks], original_sizes=inputs.original_sizes, bi
# masks has shape (num_objects, 1, H, W); we track one object, so masks[0, 0] is our (H, W) bi
```

Two things to notice: there is no `num_iters` or `threshold`, because nothing is optimized at test time, and the box is only given in frame 0, so every later frame is tracked purely from memory.

[6]:
```
def TrackSequenceSAM(seq, rect, model, processor, obj_id=1, show_progress=True):
    '''
    :param seq       : (H, W, T), sequence of grayscale frames (same format as␣
↪Q2)
    :param rect      : (4,), [x1 y1 x2 y2] coordinates of the object in the␣
↪initial frame (same as Q2.3)
    :param model     : Sam2VideoModel
    :param processor : Sam2VideoProcessor
    :param obj_id    : integer id given to the tracked object
    :return: masks   : (T, H, W) bool, mask of the tracked object in each frame
    '''
    H, W, T = seq.shape
    masks = []

    # A fresh inference session. Because we do not pass a video, the session␣
↪runs in
    # streaming mode: we feed one frame at a time, like the loop in␣
↪TrackSequence (Q2.3).
    inference_session = processor.init_video_session(inference_device=device)

    frame_iter = tqdm(range(T)) if show_progress else range(T)
    with torch.no_grad():
        for i in frame_iter:
            frame_rgb = to_rgb_frame(seq[:, :, i])

            # ===== your code here! =====
            inputs = processor(images=frame_rgb, return_tensors='pt').to(device)
            if i == 0:
              processor.
↪add_inputs_to_inference_session(inference_session=inference_session,␣
↪frame_idx=0, obj_ids=obj_id,input_boxes=[[[rect[0], rect[1], rect[2],␣
↪rect[3]]]],
                        original_size=inputs.original_sizes[0])
            out = model(inference_session=inference_session, frame=inputs.
↪pixel_values[0])
            processed_mask = processor.post_process_masks([out.pred_masks],␣
↪original_sizes=inputs.original_sizes, binarize=True)[0]
```

```
            mask = processed_mask[0, 0].detach().cpu().numpy().astype(bool)
            masks.append(mask)
            # 1. Pre-process the frame with the processor.
            # 2. On the first frame only (i == 0), add rect as a box prompt to␣
↪the inference session.
            #    Remember to pass original_size=inputs.original_sizes[0].
            # 3. Run the model on this frame.
            # 4. Post-process out.pred_masks into a full-resolution binary mask␣
↪(H, W) as a numpy bool array,
            #    and append it to masks.

            # ===== End of code =====

    masks = np.array(masks, dtype=bool)
    assert masks.shape == (T, H, W), f"Your masks {masks.shape} are not␣
↪({T}x{H}x{W})"
    return masks
```

5.2 Debug Q6.2

The snippet below runs your tracker on only the first 10 frames of the car sequence and shows the prompt box (red) together with the mask SAM 2 produced (blue) in frame 0 and in frame 9. In frame 0 the mask should fill the red box; by frame 9 the car has moved a few pixels and the mask should have moved with it.

A few tips to debug your implementation: - Check `mask.shape`, `mask.dtype` and `mask.sum()` for the first frame. If the mask is empty, the box prompt probably has the wrong shape or the wrong coordinate order (`[x1, y1, x2, y2]`, with `x` the column index). - `out.pred_masks` are *logits* at the network's low resolution; do not threshold them yourself, `post_process_masks` does the resizing and thresholding for you. - `out.object_score_logits` tells you whether the model thinks the object is visible at all (positive = visible). Printing it can help if the mask suddenly disappears.

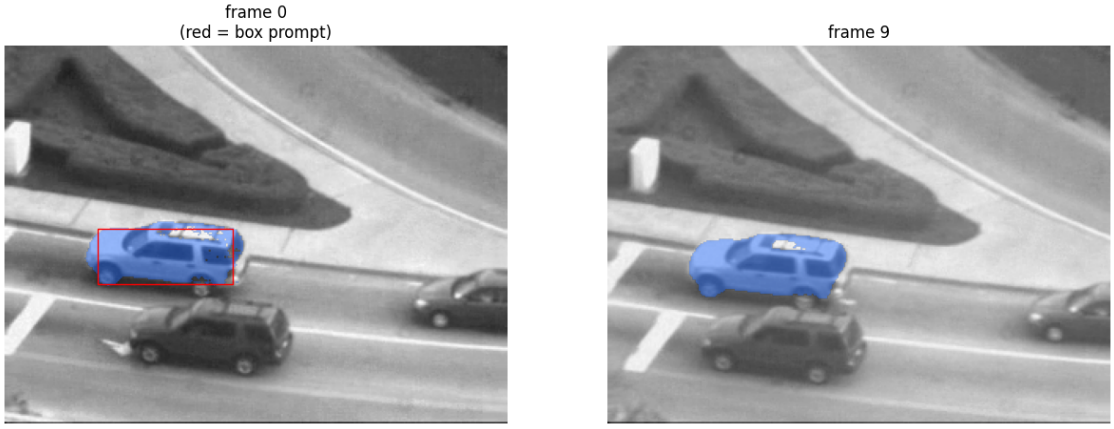
[7]:
```
seq = np.load('/content/carseq.npy')
rect = [59, 116, 145, 151]

seq_small = seq[:, :, :10]
masks_dbg = TrackSequenceSAM(seq_small, rect, model, processor)
print('mask shape:', masks_dbg.shape, ' dtype:', masks_dbg.dtype)
print('mask pixels in frame 0:', masks_dbg[0].sum(), ' in frame 9:',␣
↪masks_dbg[9].sum())

visualize_masks(seq_small, masks_dbg, [0, 9], prompt_rect=rect)
```

```
100%|      | 10/10 [00:06<00:00,  1.47it/s]
```

```
mask shape: (10, 240, 320)  dtype: bool
mask pixels in frame 0: 3072  in frame 9: 3256
```



5.2.1 Q6.2 (a) - Track Car Sequence

Run the following snippet. You should see the mask following the car in the same five frames as in Q2.3. The runtime is printed too; compare it with how long your LK tracker took in Q2.3.

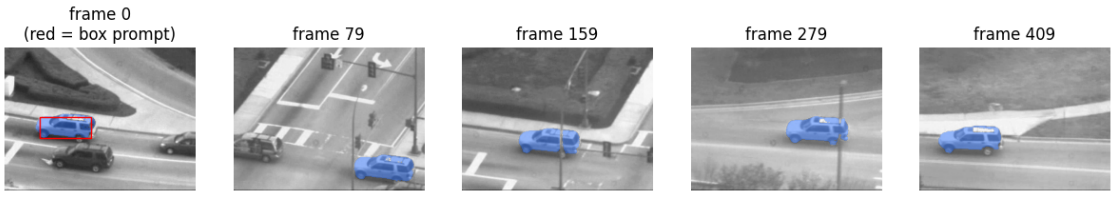
[8]:
```
seq_car = np.load('/content/carseq.npy')
rect_car = [59, 116, 145, 151]

t0 = time.time()
masks_sam_car = TrackSequenceSAM(seq_car, rect_car, model, processor)
time_sam_car = time.time() - t0
print(f'SAM 2 on carseq: {seq_car.shape[2]} frames in {time_sam_car:.1f} s '
      f'({seq_car.shape[2] / time_sam_car:.1f} frames/s)')

visualize_masks(seq_car, masks_sam_car, [0, 79, 159, 279, 409],␣
↪prompt_rect=rect_car)
```

```
100%|      | 415/415 [03:19<00:00,  2.08it/s]

SAM 2 on carseq: 415 frames in 199.2 s (2.1 frames/s)
```



5.2.2 Q6.2 (b) - Track Girl Sequence

Same as the car sequence.

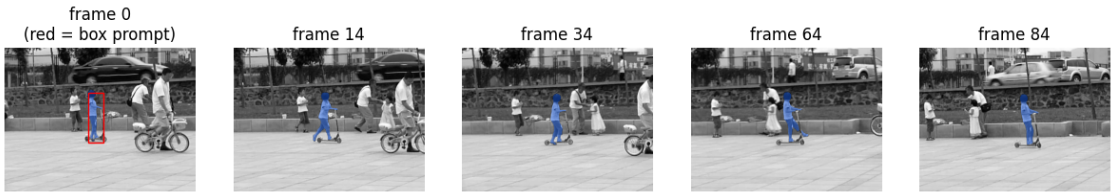
[9]:
```
seq_girl = np.load('/content/girlseq.npy')
rect_girl = [280, 152, 330, 318]

t0 = time.time()
masks_sam_girl = TrackSequenceSAM(seq_girl, rect_girl, model, processor)
time_sam_girl = time.time() - t0
print(f'SAM 2 on girlseq: {seq_girl.shape[2]} frames in {time_sam_girl:.1f} s '
      f'({seq_girl.shape[2] / time_sam_girl:.1f} frames/s)')

visualize_masks(seq_girl, masks_sam_girl, [0, 14, 34, 64, 84],␣
 ↪prompt_rect=rect_girl)
```

```
100%|      | 90/90 [00:43<00:00,  2.07it/s]

SAM 2 on girlseq: 90 frames in 43.4 s (2.1 frames/s)
```



5.3 Q6.2.1: Does the SAM 2 mask drift like the LK box did in Q2.3? Why or why not, given that SAM 2 only sees the box in frame 0?

===== your answer here! =====

SAM 2 drift less than LK box did in Q2.3. SAM 2 receives the box only in frame 0, but it maintains memory together containing with each new frame to predict the next mask.

===== end of your answer =====